

Title

ForesightX

“Microservice Based Intelligent Stock Market
Prediction & Analytics Platform”

DECLARATION BY THE CANDIDATE

We hereby declare that the Minor Project Report entitled “ForesightX: A Microservice-Based Intelligent Stock Market Prediction and Analytics Platform” submitted to the Department of Computer Science and Engineering, Jabalpur Engineering College, Jabalpur, is a record of the original work carried out by us under the guidance of **Dr. Jitendra Singh Thakur** during the academic session 2025–26.

We further declare that this work has not been submitted elsewhere for the award of any degree, diploma, or other academic qualification. The project work presented in this report is genuine and all references used have been properly acknowledged.

Aditya Pratap Singh Tomar (0201CS231003)

Stuti Jain (0201CS231100)

Date:

Place: Jabalpur

Guided By:

Dr. Jitendra Singh Thakur
Head of Department
Department of Computer Science Engineering
JEC, Jabalpur

Approved By HoD:

Dr. Jitendra Singh Thakur
Head of Department
Department of Computer Science Engineering
JEC, Jabalpur

CERTIFICATE

This is to certify that the Minor Project Report entitled “ForesightX: A Microservice-Based Intelligent Stock Market Prediction and Analytics Platform” submitted by **Aditya Pratap Singh Tomar (0201CS231003)** and **Stuti Jain (0201CS231100)** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Jabalpur Engineering College, Jabalpur, is a bona fide record of the work carried out by them under my supervision and guidance during the academic session 2025–26.

To the best of my knowledge, the work presented in this report has not been submitted elsewhere for the award of any degree or diploma.

Aditya Pratap Singh Tomar (0201CS231003)

Stuti Jain (0201CS231100)

Date:

Place: Jabalpur

Checked and Approved By:

Dr. Jitendra Singh Thakur
Head of Department
Department of Computer Science Engineering
JEC, Jabalpur

ACKNOWLEDGEMENT

We express our sincere gratitude to Dr. Jitendra Singh Thakur, Head of Department, Computer Science and Engineering Department, Jabalpur Engineering College, for his valuable guidance, continuous encouragement, and support throughout the development of this project.

We would also like to thank the faculty members of the Department of Computer Science and Engineering for providing the necessary academic environment and technical knowledge required for the successful completion of this work.

The completion of this project would not have been possible without the collaborative efforts, discussions, and continuous learning throughout the development process. Working on ForesightX helped us gain practical exposure to distributed systems, machine learning workflows, microservice architecture, frontend-backend integration, and deployment methodologies.

Finally, we would like to express our gratitude to our friends and family members for their motivation and support during the entire project duration.

Aditya Pratap Singh Tomar (0201CS231003)

Stuti Jain (0201CS231100)

INDEX

TITLE.....PAGE NO.

Title Page 1

Declaration By the Candidate 2

Certificate 3

Acknowledgement 4

Index 5

1. Introduction 9

1.1 Problem Description

1.2 Objectives of the Project

1.3 Scope of the Project

1.4 Proposed Approach

1.5 Key Features of ForesightX

2. Existing Approaches and Related Work 13

2.1 Traditional Stock Analysis Platforms

2.2 Existing Financial Analytics Systems

2.3 Research-Based Prediction Systems

2.4 Comparative Analysis

2.5 Observations from Related Work

3. Software Process Model Used 17

3.1 Agile Incremental Development Model

3.2 Justification for Model Selection

3.3 Development Lifecycle	
3.4 Phase wise Development Notes	
4. Team Formation	21
4.1 Team Structure	
4.2 Role Assignment	
4.3 Responsibilities of Team Members	
4.3 Collaboration and Communication Strategy	
5. Project Execution Plan	23
5.1 Development Timeline	
5.2 Milestones Achieved	
5.3 Week-wise Progress Summary	
5.4 Risk and Mitigation	
5.5 Gantt Chart	
5.6 PERT Chart	
6. Hardware and Software Requirements	29
6.1 Development Environment	
6.2 Deployment Environment	
6.3 Software Stack	
7. Requirement Specifications and Analysis	31
7.1 Software Requirement Specification (SRS)	
7.2 Functional Requirements	
7.3 Non-Functional Requirements	
7.4 Use Case Descriptions	
7.5 Sequence Diagram	
7.6 Collaboration Diagram	
7.7 Activity Diagram	

7.8 State Transition Diagram	
8. Design and Methodology	39
8.1 System Architecture	
8.2 Component Diagram	
8.3 Design Class Diagram	
8.4 Methodology of Authentication Service	
8.5 Methodology of Pattern Prediction Service	
8.6 Methodology of Data Service	
8.7 Methodology of Profile Service	
8.8 Methodology of Orchestration Service	
8.9 Recommendation Generation Algorithm	
8.10 Deployment Workflow	
9. Testing	53
9.1 Unit Testing	
9.2 System Testing	
9.3 API Testing	
9.4 Test Cases	
9.5 Debugging and Validation	
9.6 Testing Screenshots	
10. Conclusion	57
11. SWOT Analysis	59
11.1 Strengths	
11.2 Weaknesses	
11.3 Opportunities	
11.4 Threats	

12. References	61
Appendix	63
A. Screenshots of Modules	
B. API Endpoints	

1. Introduction

ForesightX: A Microservice-Based Intelligent Stock Market Prediction and Analytics Platform

1.1 Problem Description

Stock market analysis is often fragmented across charts, price feeds, screening tools, news sources, and separate prediction experiments. Many applications can show the user a market chart or a company snapshot, but they do not combine prediction, portfolio context, and recommendation generation into one workflow. A user therefore has to move between multiple tools and manually connect the dots, which slows down the analysis process.

This fragmentation becomes more visible when the task is not just to view a stock, but to understand the likely short-term direction, supporting signals, and confidence behind the result. A raw model output without context is difficult to interpret, while a market dashboard without prediction support often leaves the user with more data than direction. ForesightX was created to reduce that gap.

The problem addressed by the project is the lack of a unified, modular, and explainable environment for short-term stock analytics. The system aims to transform data retrieval, technical indicator computation, forecasting, and final recommendation generation into a single structured flow that is easier to use and easier to explain.

1.2 Objectives of the Project

- To design and implement a modular platform for short-term stock analysis and prediction.
- To separate the system into independent microservices so that each component can evolve without affecting the others.
- To support secure user authentication using JWT-based login and refresh tokens.
- To retrieve real-time market information and historical price data from external sources.
- To compute key technical indicators and expose them through a dedicated data service.
- To generate short-horizon forecasts using a machine learning prediction pipeline.

- To estimate prediction confidence so that the output is not presented as a rigid certainty.
- To maintain user portfolio information and basic risk context in the profile service.
- To coordinate service communication using an orchestration layer.
- To provide an AI-assisted summary that combines model output, market context, and recommendation logic.

The objectives were chosen to ensure that the project would be technically meaningful rather than merely decorative. Each objective corresponds to a concrete module or workflow in the actual system architecture, which makes the report internally consistent and easier to validate during review.

1.3 Scope of the Project

The scope of ForesightX is centered on a web-based stock analytics platform for registered users. Within this scope, the system allows a user to register, sign in, search for instruments, inspect live price movement, study historical charts, request a prediction, and view a recommendation that reflects multiple sources of information. The platform is not intended to replace a brokerage application or a trading terminal.

Instead, the project focuses on helping the user understand the market more effectively before making a personal decision. That means the system must present live market data, prediction output, technical indicators, and portfolio context in a form that is understandable without requiring the user to manually stitch together separate tools. In practical terms, it acts as a decision-support platform rather than an execution platform.

The system is designed around service ownership. Authentication, market data, prediction, orchestration, and user profile management are handled by separate microservices, each with its own database. This keeps the architecture clean and helps preserve reliability when one part of the system changes. It also makes the project suitable as a technical demonstration of distributed design, ML service integration, and workflow orchestration.

1.4 Proposed Approach

ForesightX follows a layered approach. The user interacts with the frontend, which forwards requests to the orchestration service. The orchestration service then coordinates the supporting services according to the nature of the request. If the user requests market analysis, the data

service fetches live prices, historical records, indicators, and news. If the user requests a forecast, the pattern service performs inference using the pre-trained model and returns a prediction with a confidence signal.

The orchestration layer is important because it does not merely forward requests. It consolidates outputs from different services, applies weighted reasoning, and generates a final user-facing response. This design is particularly useful in a financial analytics system where a single input can produce multiple signals that need to be interpreted together rather than in isolation.

The proposed approach also supports incremental development. A service can be built, tested, and deployed on its own before being connected to the full platform. This matched the way the project was actually developed: prediction, data handling, authentication, profile logic, frontend integration, and deployment each matured in stages instead of appearing all at once.

1.5 Key Features

The platform includes secure JWT-based authentication, Google OAuth integration, real-time and historical stock data access, technical indicator computation, ML-based short-term prediction, portfolio and risk context, and AI-assisted summary generation. The system also supports websocket-based data streaming for live updates and keeps its responsibilities divided across service-specific components.

The key design feature is not the prediction model alone, but the fact that prediction is part of a larger workflow. A stock is not simply classified or forecasted; it is analyzed in the context of market movement, user portfolio information, and orchestration-driven reasoning. That combination is what makes the system academically useful and architecturally more interesting than a single-model demo.

- Service isolation with dedicated databases.
- Reusable APIs for stock, user, and analysis workflows.
- Explainable recommendations instead of isolated numeric output.
- Local Docker-based deployment during current development.
- Preparedness for future Kubernetes and CI/CD integration.

2. Existing Approaches and Related Work

The purpose of this section is to position ForesightX among existing stock analysis tools and research systems. The comparison is not meant to claim that all prior systems are insufficient; instead, it explains how the proposed project combines useful ideas from different categories into a more integrated platform.

2.1 Traditional Stock Analysis Platforms

Traditional stock analysis platforms usually provide charting interfaces, price summaries, screeners, news feeds, and watchlists. These tools are practical because they display market information clearly and allow users to inspect trends without building their own data pipeline.

Their limitation is that they expect the user to perform most of the analytical reasoning manually. The user has to read the chart, interpret technical indicators, compare news sources, and decide what the outcome means. That approach is flexible, but it is also time-consuming and inconsistent because different users may draw different conclusions from the same data.

ForesightX borrows the convenience of visual analysis from these platforms but adds a structured prediction and orchestration layer. In other words, it keeps the ease of viewing data while reducing the amount of manual interpretation required to form a short-term market view.

2.2 Existing Financial Analytics Systems

Existing financial analytics systems often focus on dashboards, investment performance, alerts, and curated market data. They are useful for tracking portfolios and comparing stocks, and some provide technical indicators or simple news integration. However, many such systems are built as centralized applications where the data, analysis, and user logic are strongly coupled.

A centralized design can be sufficient for a small product, but it becomes harder to extend when new analysis modules or model-serving components are introduced. It can also become difficult to debug when a market data problem, a user profile issue, and a prediction failure all appear in the same application layer. The absence of clear service boundaries makes maintenance heavier than it should be.

ForesightX addresses this by separating the platform into focused services and allowing each service to own its database and logic. This improves isolation, makes the architecture easier to explain in a report, and reflects a more realistic service-based engineering style.

2.3 Research-Based Prediction Systems

Research-based systems often explore neural networks, time-series models, hybrid forecasting approaches, or attention-based architectures for stock prediction. These systems can be valuable because they show how a model behaves on historical sequences and whether certain features improve accuracy.

The weakness of many research systems is that they remain disconnected from the rest of the application ecosystem. They are typically trained in notebooks or scripts, evaluated on datasets, and discussed in terms of metrics. They may not include a live API, a user interface, portfolio context, or explainable recommendation delivery.

ForesightX includes the research value of model development but wraps it inside a usable service architecture. The model is not left as a theoretical artifact. It becomes part of a deployable prediction service that can respond to live user input and feed results into the broader decision flow.

2.4 Comparative Analysis

A meaningful comparison must consider data access, prediction capability, architecture style, and the degree to which the result is understandable to the user. The comparison below summarizes how ForesightX differs from traditional platforms, analytics dashboards, and standalone research systems.

Aspect	Traditional Platforms	Analytics Systems	Research Systems	ForesightX
Data access	Manual or API view	Dashboard-focused	Dataset-oriented	Live, historical, and cached
Prediction	Usually absent	Limited warnings	Model-based	Integrated short-term forecast

Architecture	Monolithic	Centralized	Experiment-driven	Service-oriented
Explainability	Minimal	Moderate	Low for end users	Traceable orchestration
User context	Watchlists	Portfolio views	Rarely included	Portfolio-aware analysis
Deployment readiness	Production apps	Semi-structured	Often offline	Containerized prototype
Extensibility	Depends on vendor	Moderate	High in code	High across services

From this comparison it becomes clear that ForesightX is not a simple charting application and not only a model demonstration either. It combines practical market access with a distributed service structure, which makes the project more representative of an actual software system than a small standalone experiment.

The comparison also shows why the orchestration layer matters. When several different signals are available, the system needs a way to combine them without forcing the user to inspect each raw result independently. That design choice gives the platform a functional advantage in readability and a structural advantage in maintainability.

In practical terms, the comparison shows a shift from data display to decision support. That shift is the main reason the project deserves to be documented as a proper software system rather than only as an ML notebook with a user interface.

2.5 Observations from Related Work

The main observation from related systems is that most platforms specialize in one slice of the problem. Some focus on monitoring, some on charting, some on prediction, and some on portfolio display. ForesightX attempts to place those slices inside one architecture so that the user does not have to combine them manually.

Another observation is that when prediction is isolated from the user workflow, the result often becomes difficult to use. A forecast is useful only when the user can see its context. The project

therefore uses orchestration, contextual data, and explanation together instead of treating the model output as a standalone answer.

3. Software Process Model Used

The development of ForesightX followed the Agile Incremental Development Model. This process model was suitable because the project included multiple modules, parallel work, and frequent integration points. Rather than attempting to design every component completely before coding, the team developed the system in small functional increments and refined it after each stage.

Agile was also appropriate because the project included both infrastructure-heavy and user-interface-heavy tasks. Backend services, model logic, and deployment work could proceed in one stream while frontend design and integration progressed in another. That reduced idle time and made the project easier to control within the semester schedule.

3.1 Agile Incremental Development Model

In the incremental model, each iteration delivers a working part of the system. For ForesightX, the early iterations focused on defining the problem, deciding the service boundaries, and establishing the prediction direction. Subsequent iterations introduced authentication, market data retrieval, model serving, profile management, and finally orchestration and integration.

This model is particularly useful when the product can be divided into independent yet related modules. A failure or change in one service does not force the whole project to restart. It is easier to test one increment at a time and confirm that it behaves correctly before moving to the next.

3.2 Justification for Model Selection

- The project consists of multiple services that can be created and validated in parallel.
- The prediction model required repeated refinement of features, sequence design, and inference flow.
- The user interface could be developed while backend APIs were being stabilized.
- Integration risks were reduced because each service could be verified independently.
- The team needed a flexible model that allowed partial delivery and later refinement.
- Documentation and testing could follow the evolution of the software rather than wait until the very end.
- The Agile model matched the actual working pattern of the team. One member could focus on the backend and prediction stack while the other member worked on the

interface and integration support. This parallelism made it possible to keep the project moving forward even when one layer required more time than the others.

- It also made the report easier to justify academically. Rather than describing the project as a single coding sprint, the document can show a series of planned iterations, each with its own goal and completion state. That is more credible and more informative.

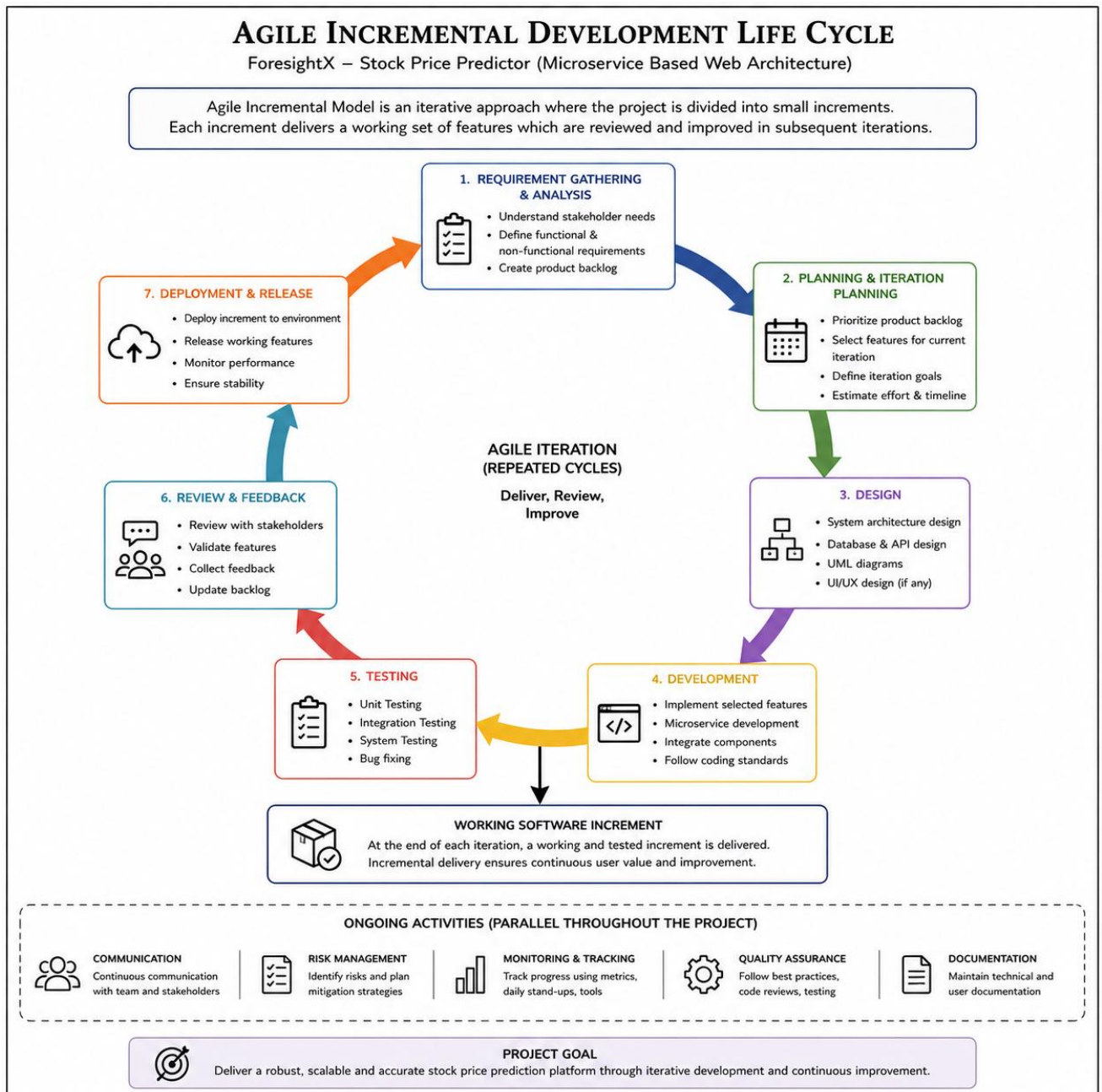
3.3 Development Cycle

The development cycle began with requirement analysis and research. The team studied the problem domain, identified the need for a stock analytics workflow, and decided that a distributed architecture would fit the scope better than a monolithic design. The outcome of this stage was a clear understanding of the project's purpose, service boundaries, and data flow.

The second phase focused on ML architecture and backend planning. The prediction service required a proper feature contract, a training workflow, and an inference endpoint that could be used consistently by the rest of the platform. At the same time, the core services were planned so that their responsibilities would remain independent.

The third phase covered implementation. Services were built one by one, tested locally, and then connected into a single workflow. The frontend was designed and integrated in parallel. After the main functionality was in place, the system moved toward deployment preparation, testing, and documentation.

A typical cycle for ForesightX therefore follows the order: define problem, shape service boundaries, implement service logic, validate the interfaces, integrate the workflow, and refine the output. That cycle repeats at a smaller level every time a service or model component is improved.



Because the system is microservice-based, the development cycle also includes repeated integration checks. If the data service changes, the orchestration layer needs validation. If the prediction output changes, the UI and recommendation flow need to be checked.

3.4 Phase-wise Development Notes

- I. Requirement analysis defined the user role, the problem statement, and the broad output expected from the system. This stage prevented unnecessary scope expansion and gave the team a clear target for the rest of the semester.
- II. Architecture design established the service boundaries and identified which responsibilities belonged to authentication, data, prediction, orchestration, and profile management. This reduced overlap and made the microservice logic easier to manage.
- III. Model planning determined the sequence structure, feature contract, and confidence-oriented prediction flow. This was essential because the ML service had to produce usable outputs rather than only raw numerical values.
- IV. Backend implementation added the service endpoints, database interactions, and coordination logic. Each service was treated as an independent component with its own life cycle and deployment assumptions.
- V. Frontend integration connected the application views to the backend APIs so that the system could be used through a browser rather than only through direct service calls.
- VI. Testing and refinement validated the system behavior after integration. Errors were fixed in response to service interactions, not just isolated unit logic.

4. Team Formation

The project was completed by a two-member team. The distribution of work was planned so that both members could contribute continuously without waiting for one another to finish an entire module. This was important because the project had both technical depth and presentation requirements, and both needed attention throughout the semester.

4.1 Team Structure

The team structure followed a **lead-and-support approach** for different modules, but the work remained collaborative. Technical ownership was assigned according to the nature of the task rather than according to a rigid hierarchy. In a microservice project, this is a practical way to work because different modules often require different kinds of effort.

For example, backend service design and prediction logic demanded concentration on architecture, API behavior, and deployment considerations. The user interface required visual design, interaction flow, and integration timing. Testing and documentation could then be shared because they benefited from both viewpoints.

4.2 Role Assignment

Team Member	Primary Role	Major Responsibilities	Justification
Aditya Pratap Singh Tomar 0201CS231003	Backend, ML, Orchestration, Deployment	Designed and implemented service logic, model integration, service communication, Docker workflow, and orchestration behavior.	This role matched the architecture-heavy and implementation-heavy parts of the project, especially where service integration and prediction logic were central.
Stuti Jain 0201CS231100	Frontend, UI Integration, Testing,	Created and refined the user interface, supported frontend integration, assisted in	This role matched the presentation and validation needs of the project, especially where

	Documentation Support	testing, and contributed to documentation preparation.	user flow and final usability were important.
--	--------------------------	---	--

4.3 Responsibilities of Team Members

Aditya Pratap Singh Tomar handled the technical backbone of the project. This included defining the microservice structure, creating the backend APIs, preparing the ML workflow, linking services through orchestration, and organizing the deployment-oriented elements. This responsibility was suitable because the service architecture and model-serving logic required careful coordination and a strong understanding of backend behavior.

Stuti Jain handled the interface layer and the supporting validation work. While the backend was being developed, the frontend design could progress in parallel so that the application would not remain usable only from the server side. Later, she also contributed to integration checks and documentation support, which helped keep the final presentation coherent and balanced.

The chosen responsibilities ensured that both team members worked on meaningful parts of the project rather than splitting the work into isolated fragments with no overlap. The result was better communication, faster feedback, and smoother integration. In a report, this also demonstrates that the team was organized according to capability and project need rather than convenience alone.

4.4 Collaboration and Communication Strategy

The team used a simple but effective collaboration pattern: discuss the architecture first, divide the next implementation step, review the outcome, and then move to the next increment. This prevented the project from becoming a collection of disconnected features. Every major change in the backend had a corresponding review in the frontend or in the orchestration flow.

This approach also helped during troubleshooting. If a service response changed, it could be traced back to the owning module rather than to the whole application. That is one of the practical benefits of microservice design and one of the reasons the project is useful as a reportable engineering exercise.

5. Project Execution Plan

The project execution plan began in the middle of January 2025 and continued until the end of April 2025. The schedule was structured so that the team could complete the major development work within the semester window, while leaving a small margin for final adjustment, testing, and documentation. The project was therefore planned as a sequence of controlled stages rather than a rushed last-minute build.

The execution plan reflects the actual rhythm of the project. Some tasks were sequential because they depended on earlier outputs. Others were parallel because they could be developed independently. That is exactly the sort of thing a good project plan should show: which parts waited for other parts, and which parts sensibly moved together.

5.1 Development Timeline

The initial week was devoted to requirement analysis, domain study, and planning. Once the direction was clear, the team spent the next few weeks building the ML architecture and starting the backend structure. During that period, the frontend design also moved ahead in parallel, which helped reduce waiting time later during integration.

The backend microservices were then completed in phases. The pattern service came first because the prediction contract was central to the project. After that, the orchestration, data, and authentication services were completed, and finally the profile service was finalized. Once the core services were stable, the frontend integration, deployment preparation, and testing could happen more efficiently.

Phase	Approx. Duration	Activity Summary	Outcome
Requirement Analysis and Research	1 week	Studied the problem domain and planned the system structure.	Project scope defined.
ML Architecture and Model Planning	2.5 to 3 weeks	Prepared the feature contract, model direction, and inference flow.	Prediction pipeline designed.

Backend Microservice Development	About 1 month	Implemented auth, data, pattern, orchestration, and profile services.	Core system developed.
UI Design and Frontend Integration	Parallel work	Designed the interface and connected it with the backend services.	User-facing layer completed.
Testing and Validation	Final stage	Performed unit and system-level checks on the implementation.	Behavior verified.
Deployment Preparation and Documentation	Final week(s)	Containerized the system and documented the project.	Submission-ready draft prepared.

5.2 Milestones Achieved

The first milestone was the confirmation of the project idea and the service-based architecture. This milestone was important because it prevented the team from drifting into an overly broad design. The second milestone was the completion of the ML model architecture, which gave the team a stable basis for the prediction service.

The third milestone was the successful development of the microservices, which turned the design into a working platform. The fourth milestone was frontend integration. At that point the project started behaving like a complete application rather than a set of APIs. The fifth milestone was deployment preparation using Docker.

The sixth milestone was documentation, which captured the work in academic form for submission. The seventh planned milestone is the expansion toward Kubernetes and CI/CD, which will be treated as a later enhancement rather than part of the current submission.

Milestone	Target Stage	Status	Significance
Problem and scope finalization	Initial phase	Completed	Defined the direction and prevented unnecessary expansion.
ML pipeline architecture	Early build	Completed	Established the forecasting foundation.
Service implementation	Core build	Completed	Produced the auth, data, pattern, orchestration, and profile layers.
Frontend integration	Integration phase	Completed	Connected the user interface to the backend workflow.
Dockerized deployment	Final delivery phase	Completed	Enabled consistent local execution.
Testing and documentation	Submission phase	Completed	Completed verification and reporting.
Kubernetes and CI/CD	Future phase	Planned	Reserved for next-stage expansion after submission.

The Gantt chart and PERT chart will later be inserted in the report to visualize this plan. Their role is to show the timeline and dependency structure, but the written description here already explains the reasoning behind the schedule and the order in which the tasks were executed.

To make the execution plan easier to review, the next addition is a week-wise summary of the major actions taken by the team. This is useful because Gantt charts show duration, while narrative text explains why the durations were assigned the way they were.

5.3 Week-wise Progress Summary

Weeks	Focus	Primary Work	Result
1	Planning	Requirement study, research, and scope freeze	Project direction finalized
2-4	ML Architecture	Feature design, model selection, and pipeline planning	Prediction approach established
3-6	Frontend/UI	Layout design and page structure	User interface foundation ready
5-8	Microservices	Auth, data, and pattern service coding	Core API services completed
7-9	Orchestration	Workflow coordination and response composition	Integrated analysis flow formed
8-10	Profile	Portfolio and risk logic	User context support added
10-11	Integration	Frontend API hookup and workflow tests	End-to-end paths verified
11-12	Deployment	Dockerization and local environment tuning	Containerized execution achieved

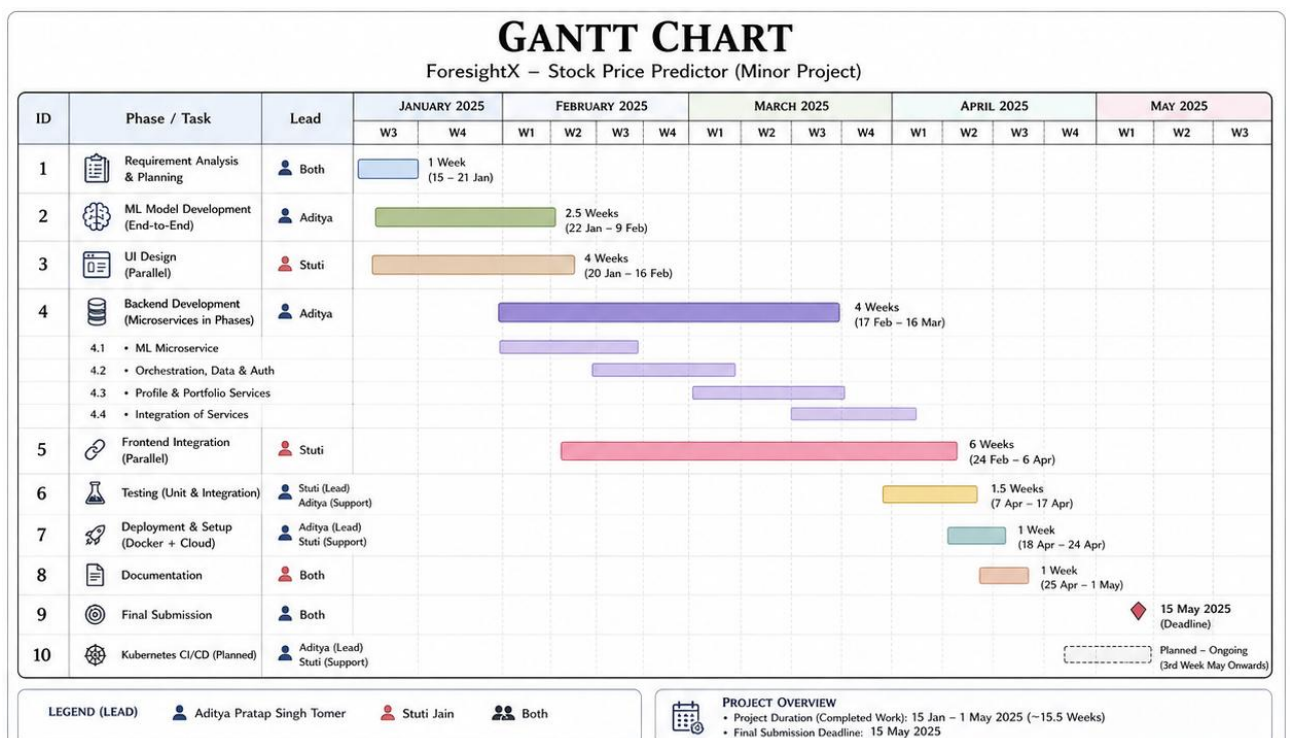
12-13	Documentation	Report drafting, screenshots, and cleanup	Submission draft prepared
-------	---------------	---	---------------------------

5.4 Risks and Mitigation

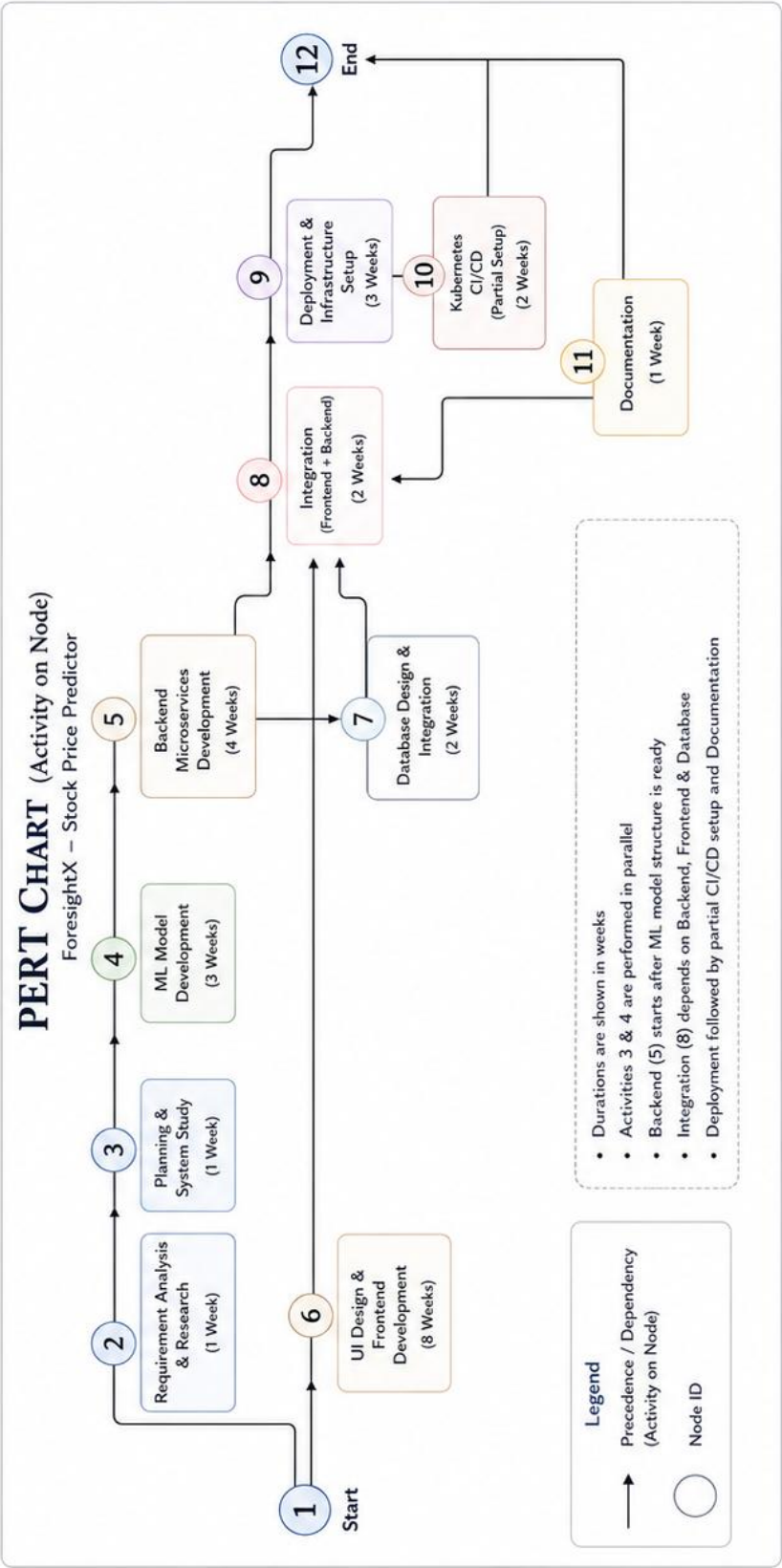
The main risk during execution was service dependency. If one microservice changed its response format, another part of the system could fail during integration. To reduce this risk, the services were developed incrementally and tested at each stage rather than leaving all verification for the end. This allowed the team to catch issues before they spread across the architecture.

A second risk was time pressure near the submission stage. That was controlled by parallel work allocation, which ensured that frontend development, backend service completion, and documentation did not all happen in a single rushed cycle. The result was a more stable and more reviewable project output.

5.5 Gantt Chart



5.6 PERT Chart



6. Hardware and Software Requirements

6.1 Development Environment

The development environment of ForesightX was designed to support backend microservice development, machine learning experimentation, frontend integration, and containerized execution. Since the project combines distributed systems and ML workflows, the development setup needed to handle multiple running services, local databases, and model training operations simultaneously.

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or above
RAM	16 GB Recommended
Storage	512 GB SSD
Operating System	Windows 11 / Ubuntu Linux
GPU	Optional NVIDIA GPU for ML Training
Browser	Google Chrome / Firefox

6.2 Deployment Environment

The current implementation supports local Docker-based deployment where each microservice executes in an isolated container. This approach ensures consistency across development and execution environments. The future deployment plan includes migration toward a Kubernetes-based orchestration workflow using Amazon EKS and CI/CD-driven automated deployment.

- Docker-based containerization for service isolation
- AWS EC2 infrastructure for deployment
- Future Kubernetes orchestration through EKS
- Planned CI/CD integration using GitHub Actions
- Independent deployment lifecycle for each microservice
- Scalable cloud-ready architecture

6.3 Software Stack

ForesightX combines modern web development technologies, distributed backend services, machine learning frameworks, and MLOps tooling. The software stack was selected to ensure modularity, reproducibility, and maintainability.

Category	Technology	Purpose
Frontend	React, Vite, Tailwind CSS	User interface and visualization
Backend	Python FastAPI	Microservice APIs
Database	PostgreSQL (NeonDB)	Persistent data storage
Authentication	JWT + OAuth	Secure authentication
ML Frameworks	PyTorch, Scikit-learn	Model training and inference
Data Processing	NumPy, Pandas	Feature engineering
MLOps	MLflow, DVC, DAGsHub	Experiment tracking and versioning
Deployment	Docker	Containerization
Future Infrastructure	Kubernetes (EKS)	Scalable orchestration

7. Requirement Specifications and Analysis

7.1 Software Requirement Specification (SRS)

The Software Requirement Specification defines the behavior, architecture, performance expectations, and service interaction flow of ForesightX. The system is designed as a modular stock analysis and prediction platform where each microservice handles a specific responsibility. The SRS ensures that the requirements remain measurable, traceable, and aligned with the project objectives.

The platform supports authenticated users who can search stocks, retrieve live market data, analyze trends, view predictions, maintain portfolios, and receive recommendation summaries generated through orchestration logic.

7.2 Functional Requirements

- The system shall allow users to register and login securely.
- The authentication service shall generate JWT access and refresh tokens.
- The system shall support Google OAuth authentication.
- The data service shall retrieve live and historical stock market data.
- The system shall provide technical indicators such as moving averages and RSI.
- The pattern prediction service shall generate next-hour and next-day forecasts.
- The prediction system shall return confidence scores and trend direction.
- The orchestration service shall aggregate outputs from multiple services.
- The profile service shall maintain portfolio and risk information.
- The system shall provide websocket-based live updates.
- The platform shall provide AI-generated recommendation summaries.

7.3 Non-Functional Requirements

The non-functional requirements of ForesightX are particularly important because the platform depends on distributed communication, live data fetching, prediction latency, and user responsiveness. The project was designed to maintain acceptable response times while handling multiple interconnected workflows.

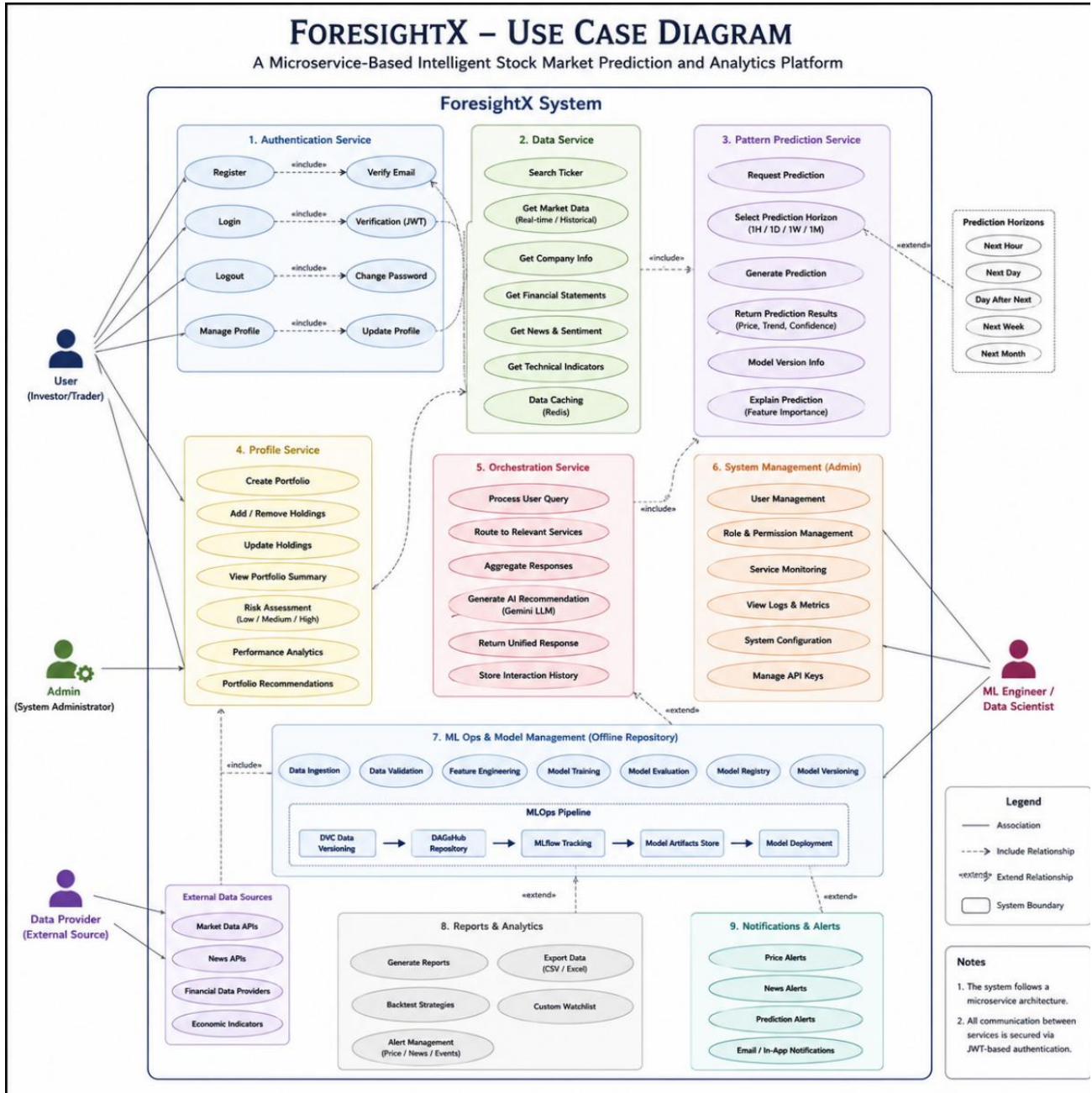
Requirement	Description	Target
Performance	One-year stock chart retrieval latency	Approximately 3-4 seconds
Prediction Response	Prediction generation latency	Within a few seconds
Scalability	Independent service scaling	Microservice isolation
Reliability	Service fault isolation	Failure of one service should not crash entire system
Security	JWT-based authentication	Secure tokenized sessions
Maintainability	Independent codebases	Service-wise modularity
Availability	Containerized deployment	Consistent execution
Extensibility	Future Kubernetes support	Cloud scalability

The observed latency of approximately three to four seconds while loading one-year chart data is considered acceptable for the current architecture and is comparable to the loading time observed in large-scale consumer financial platforms. This behavior results from network-bound data retrieval, chart rendering, and data processing overhead. To reduce repetitive fetch operations, the platform uses caching strategies within the data service.

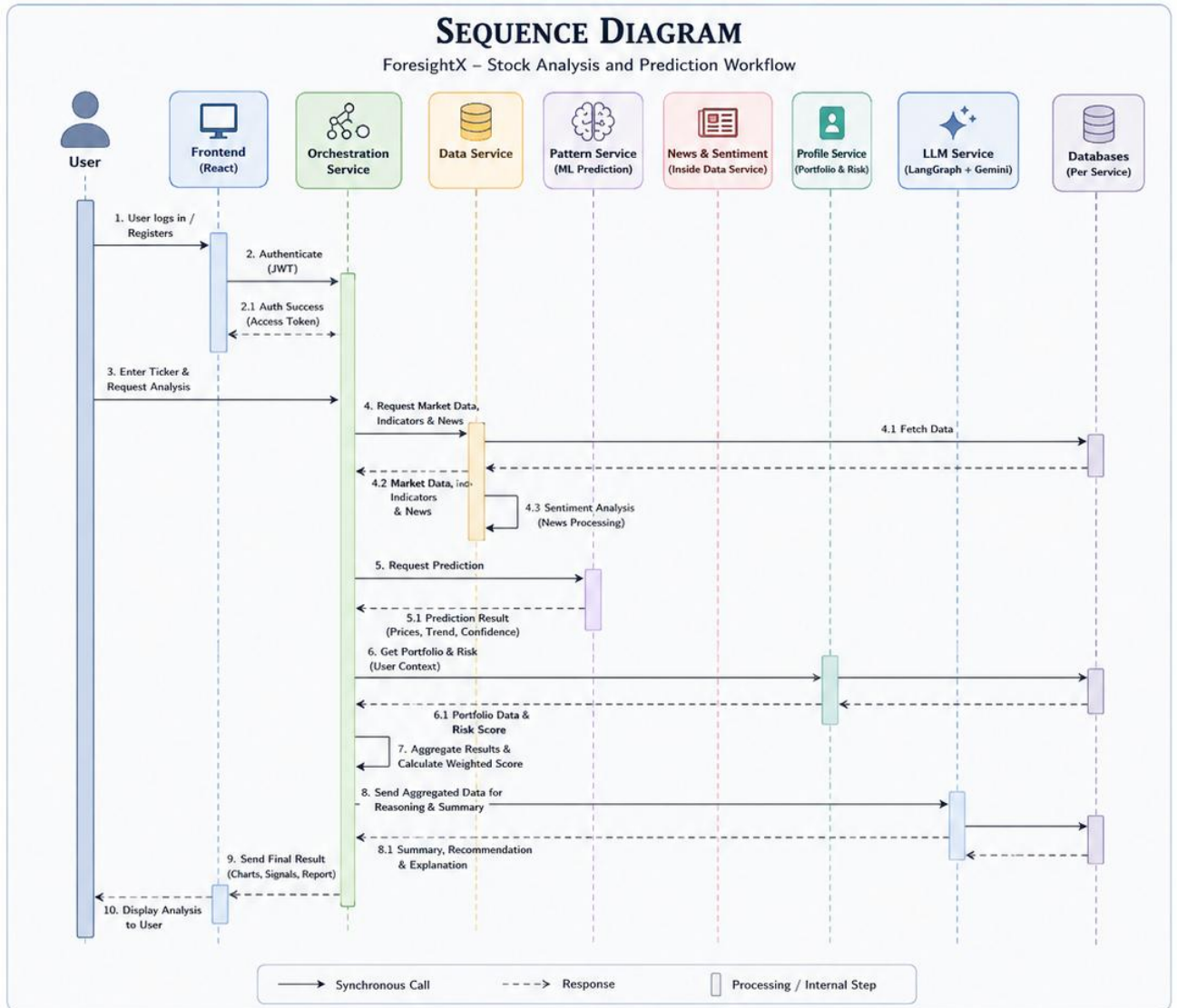
The system was also designed with maintainability in mind. Since every service exists as an independent module with separate responsibilities and databases, future changes can be implemented without affecting the complete application. This is particularly useful in prediction-oriented systems where the ML workflow evolves more rapidly than the UI layer.

7.4 Use Case Description Diagram

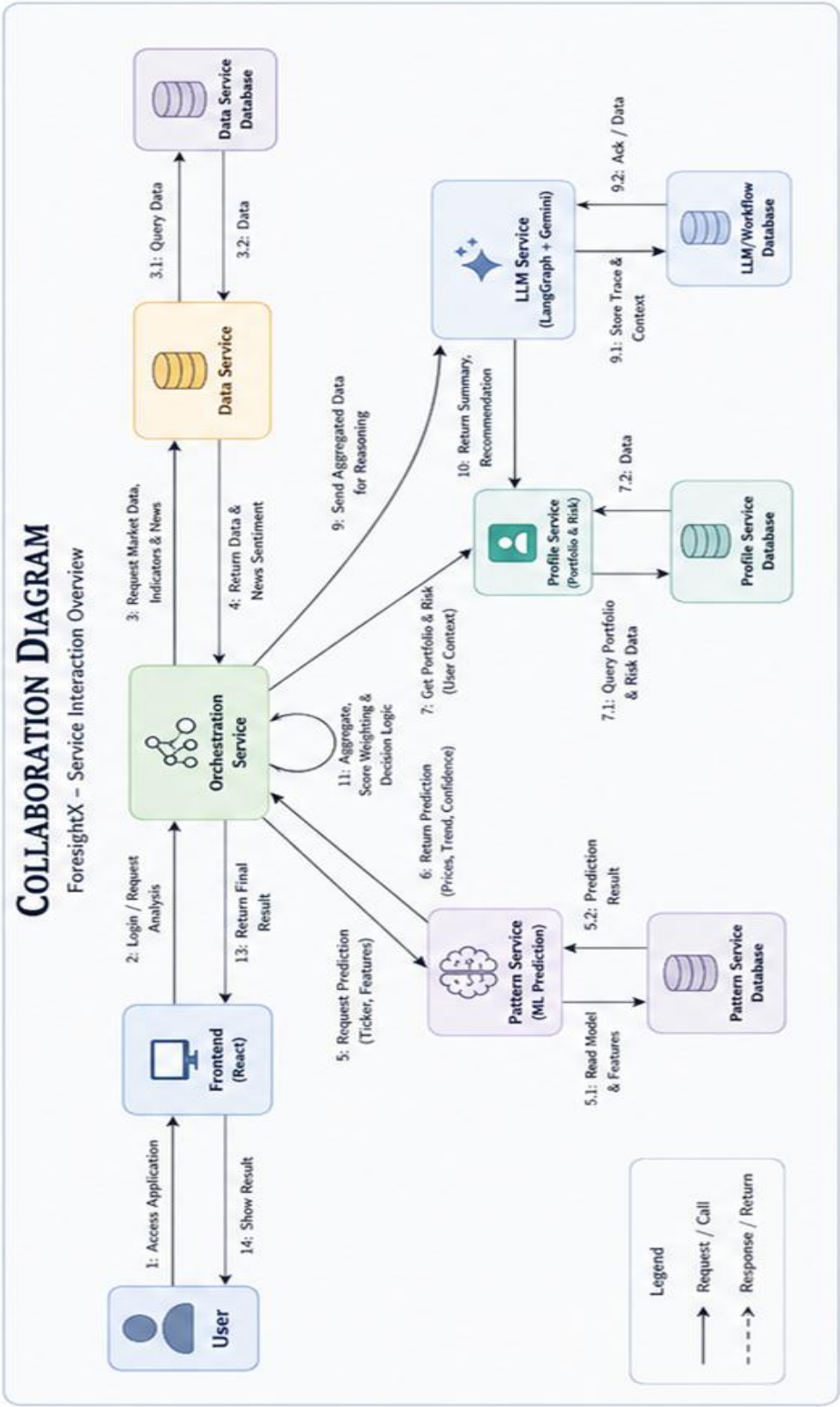
The following section contains the use case description diagram representing the workflow and interaction flow of the ForesightX platform.



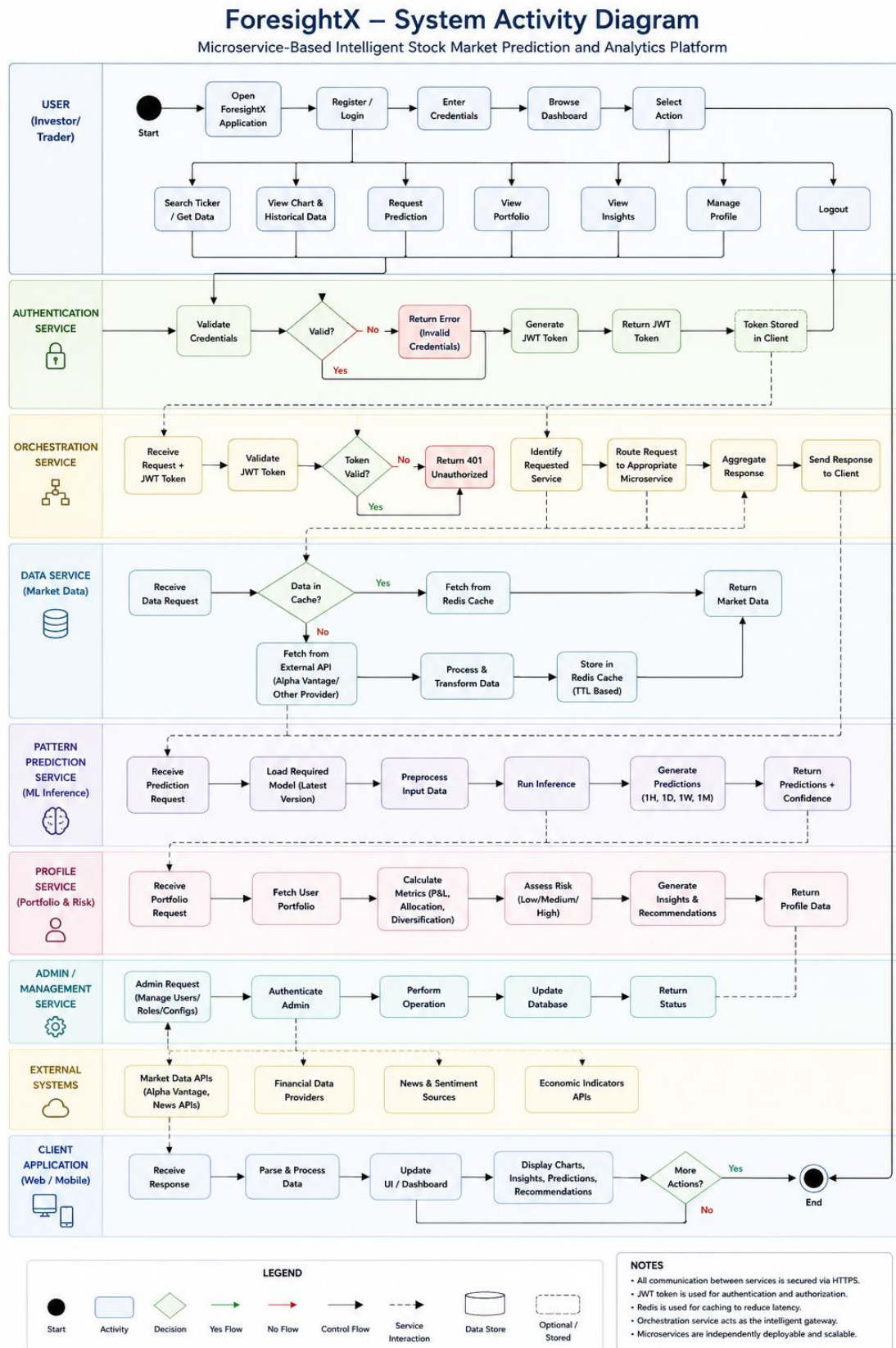
7.5 Sequence Diagram



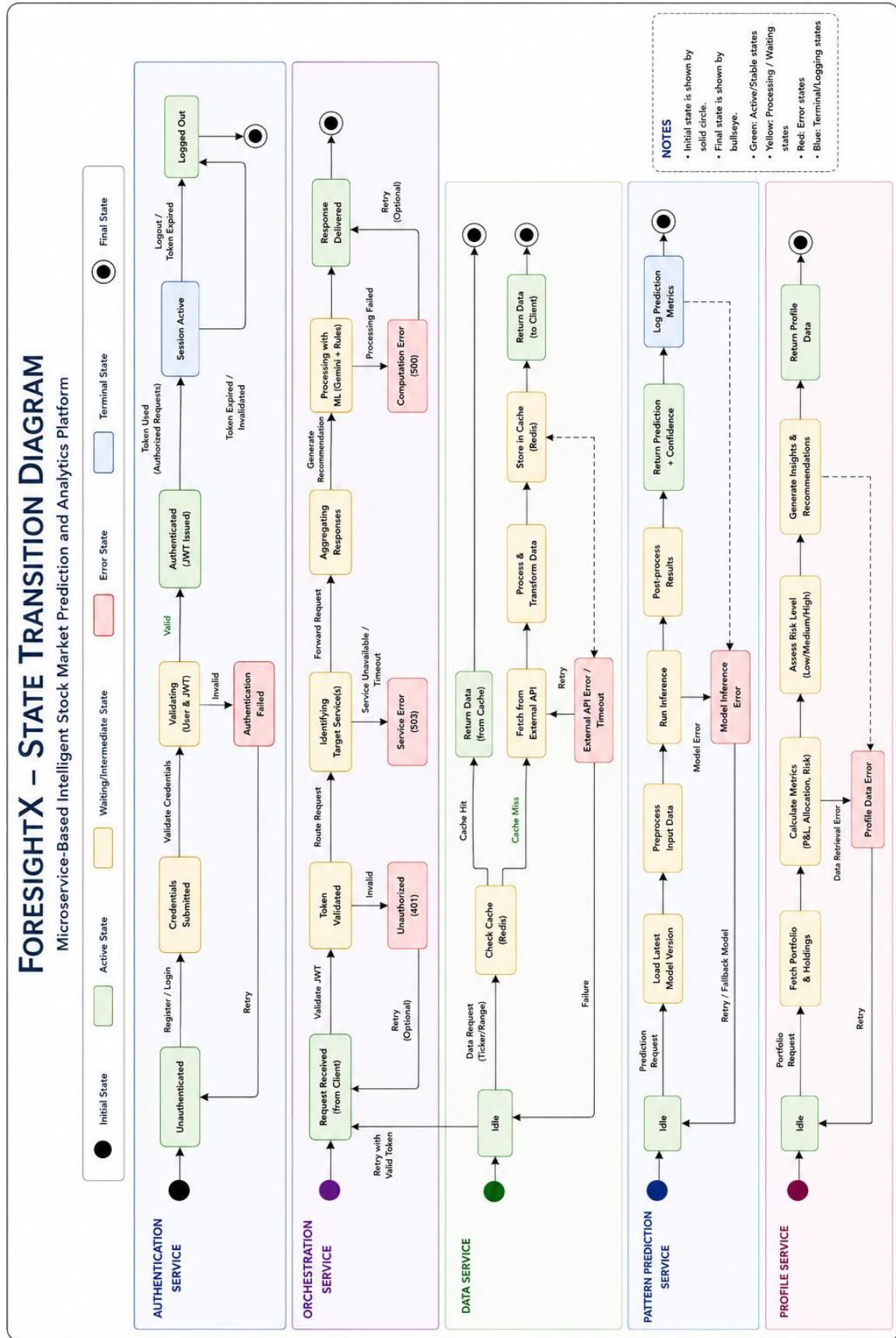
7.6 Collaboration Diagram



7.7 Activity Diagram

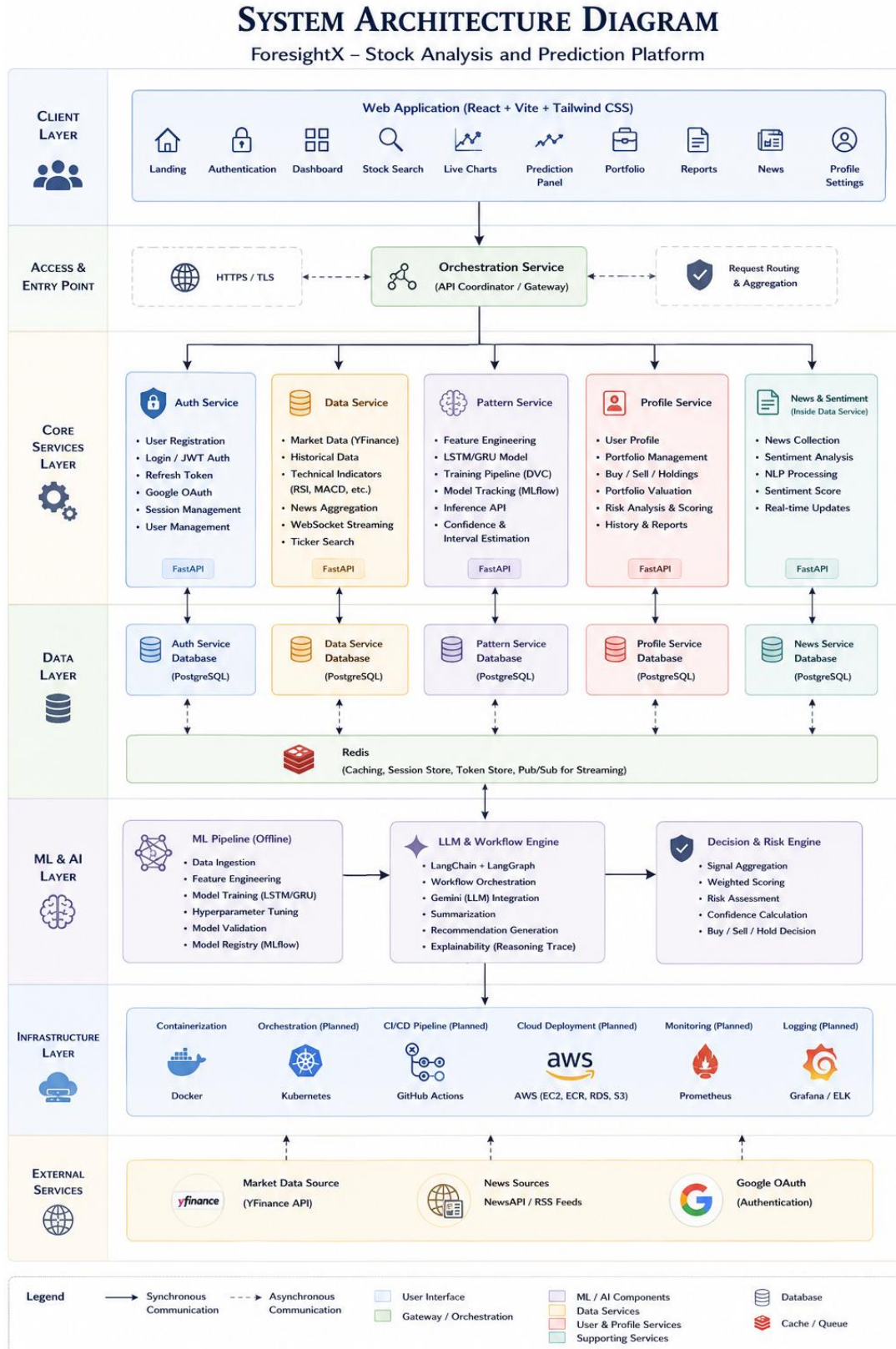


7.8 State Transition Diagram

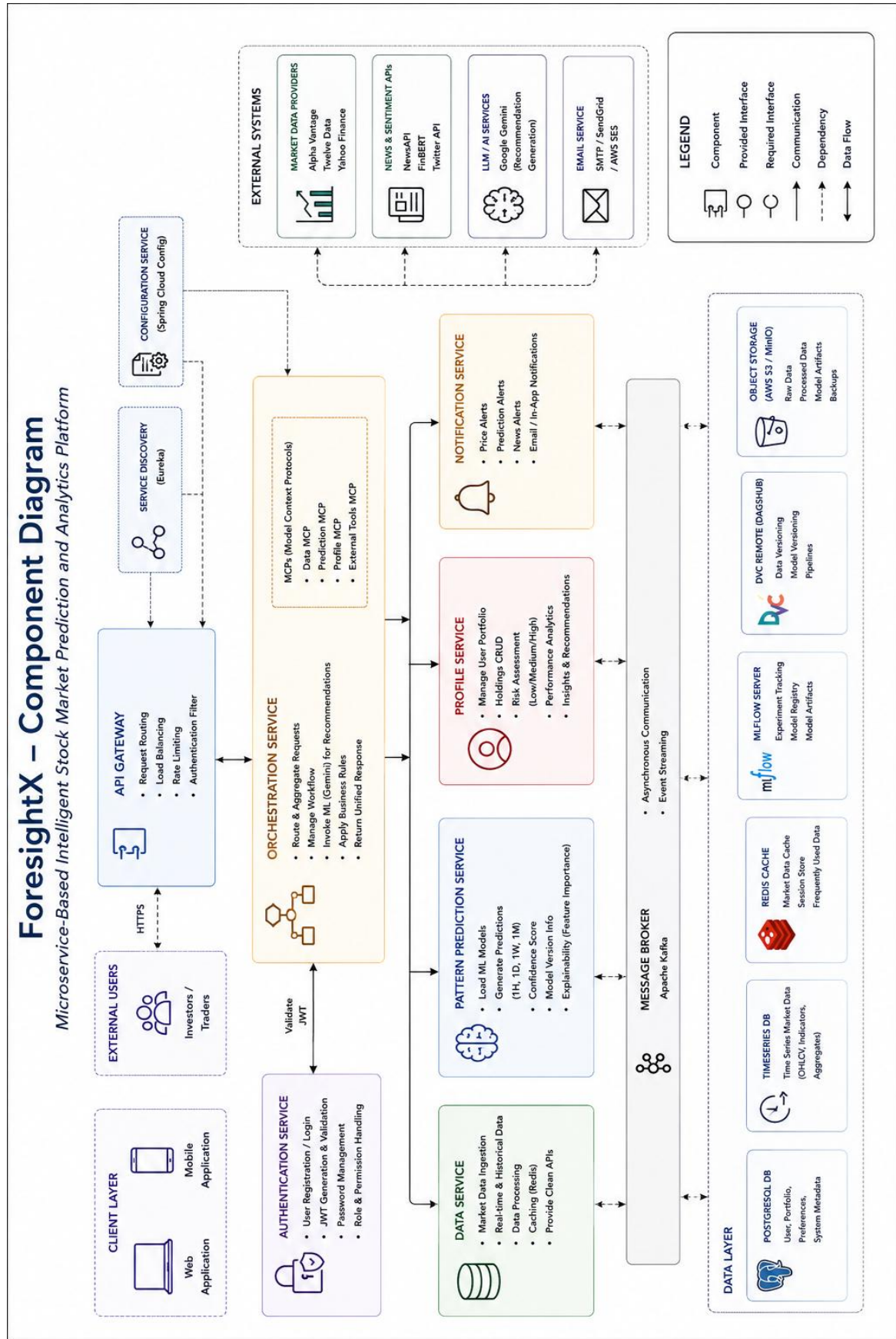


8. Design and Methodology

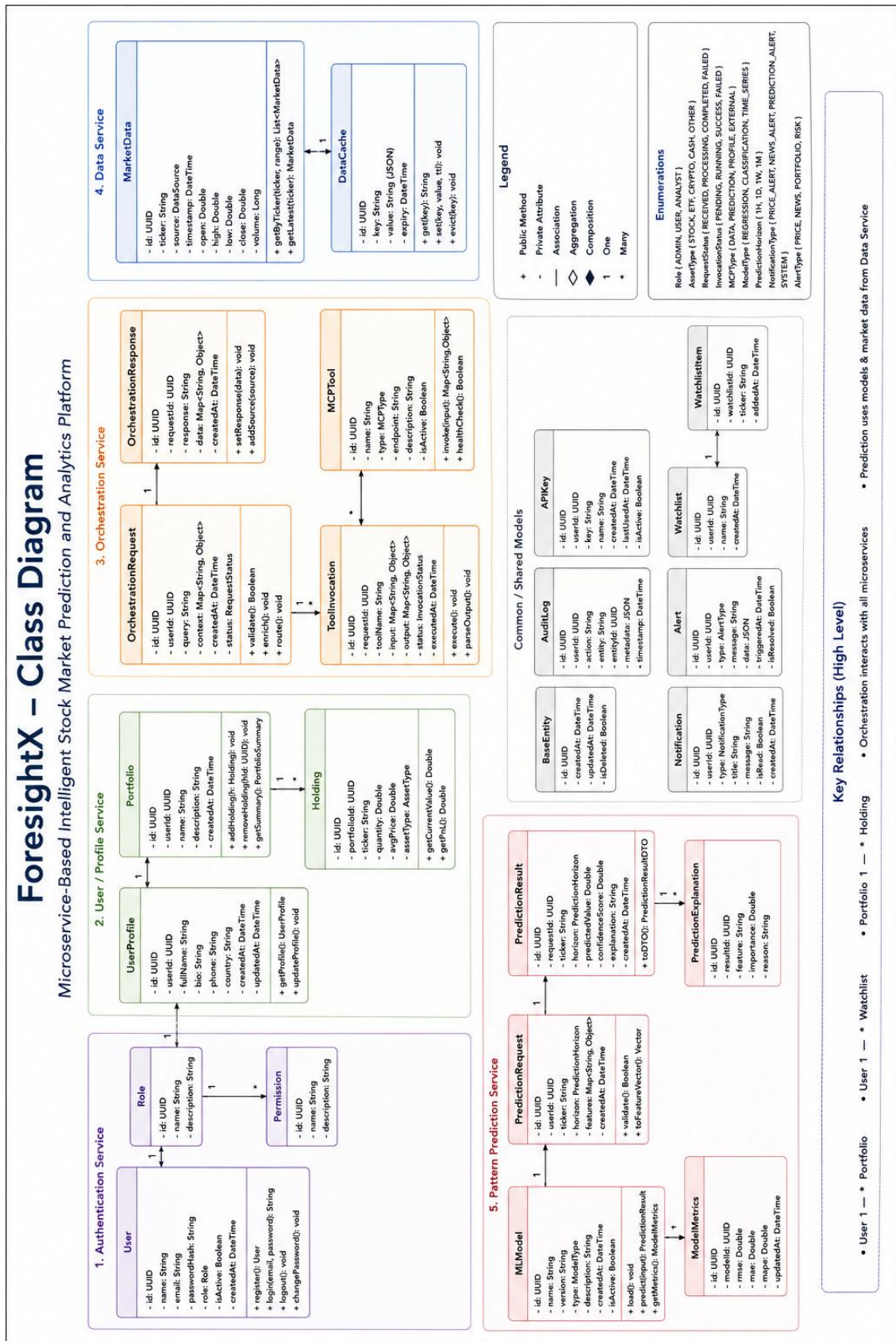
8.1 System Architecture



8.2 Component Diagram



8.3 Design Class Diagram



8.4 Methodology of Authentication Service

The authentication service is responsible for secure user registration, login, token refresh, and user identity management. The service follows JWT-based authentication, where access tokens and refresh tokens are issued after successful authentication. The service also supports Google OAuth authentication for simplified onboarding.

JWT authentication was selected because it provides stateless authentication suitable for distributed microservice architectures. Since every service can independently verify the token signature, centralized session storage is not required. This improves scalability and reduces dependency overhead.

- Stateless authentication mechanism
- Independent token validation across services
- Reduced session management overhead
- Improved scalability for distributed systems
- Refresh token mechanism for secure session continuation

8.5 Methodology of Pattern Prediction Service

The pattern prediction service is one of the most important components of ForesightX. The service is divided into two major layers: the offline ML pipeline layer and the online inference endpoint layer. This separation ensures that training and prediction responsibilities remain independent.

The offline pipeline is responsible for dataset preparation, preprocessing, feature engineering, model training, experiment tracking, and model version management. The project integrates DVC, DAGsHub, and MLflow to support reproducible machine learning workflows. DVC manages dataset versioning and pipeline reproducibility, MLflow tracks model experiments and metrics, while DAGsHub integrates experiment management and repository synchronization.

The use of an MLOps-oriented workflow improves reproducibility and future retraining capability. Instead of treating the model as a static artifact, the platform treats it as a continuously improvable component.

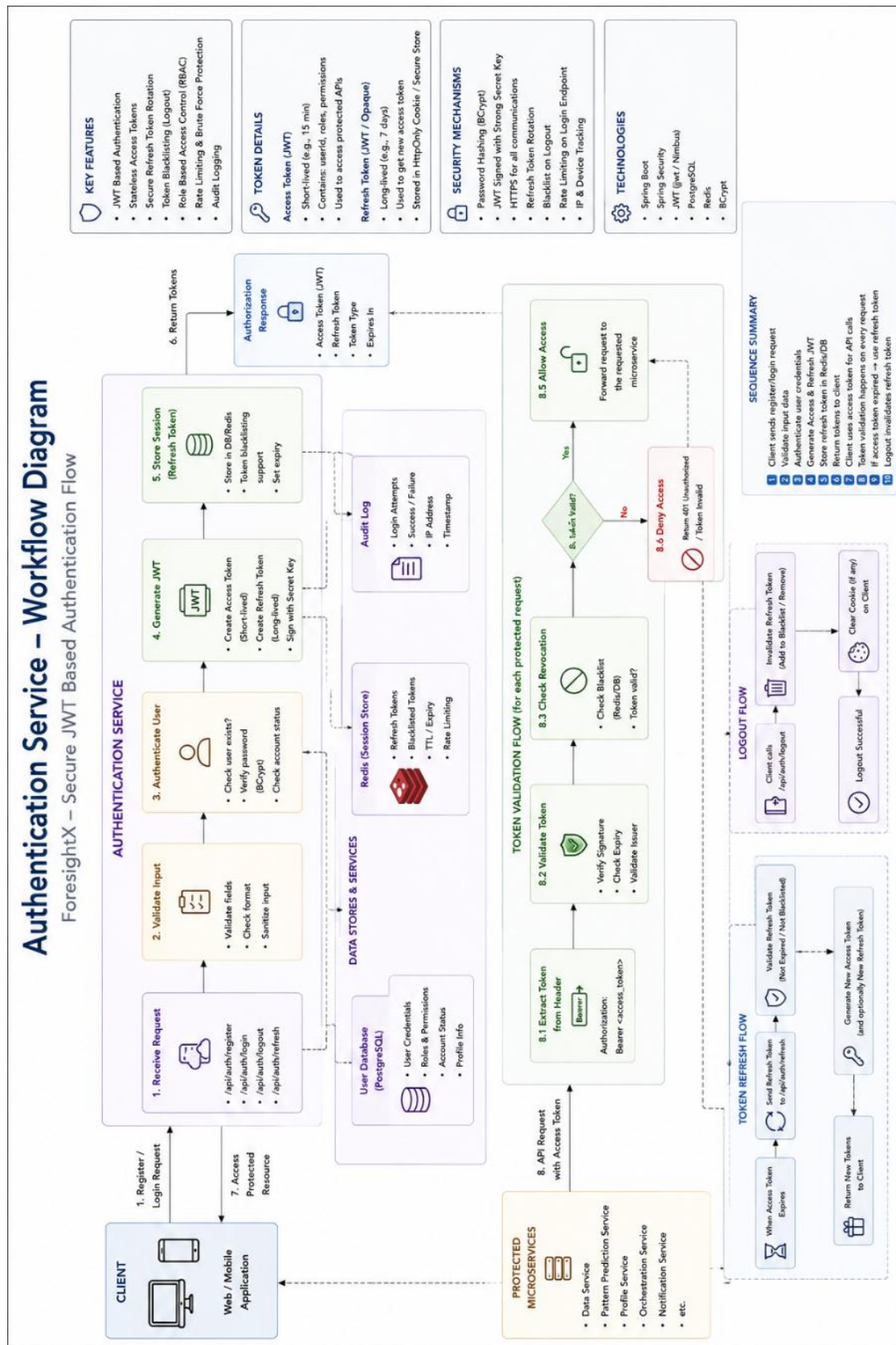
- Dataset preprocessing and normalization
- Feature engineering pipeline

- Experiment tracking using MLflow
- Dataset versioning using DVC
- Repository synchronization using DAGsHub
- Model retraining support

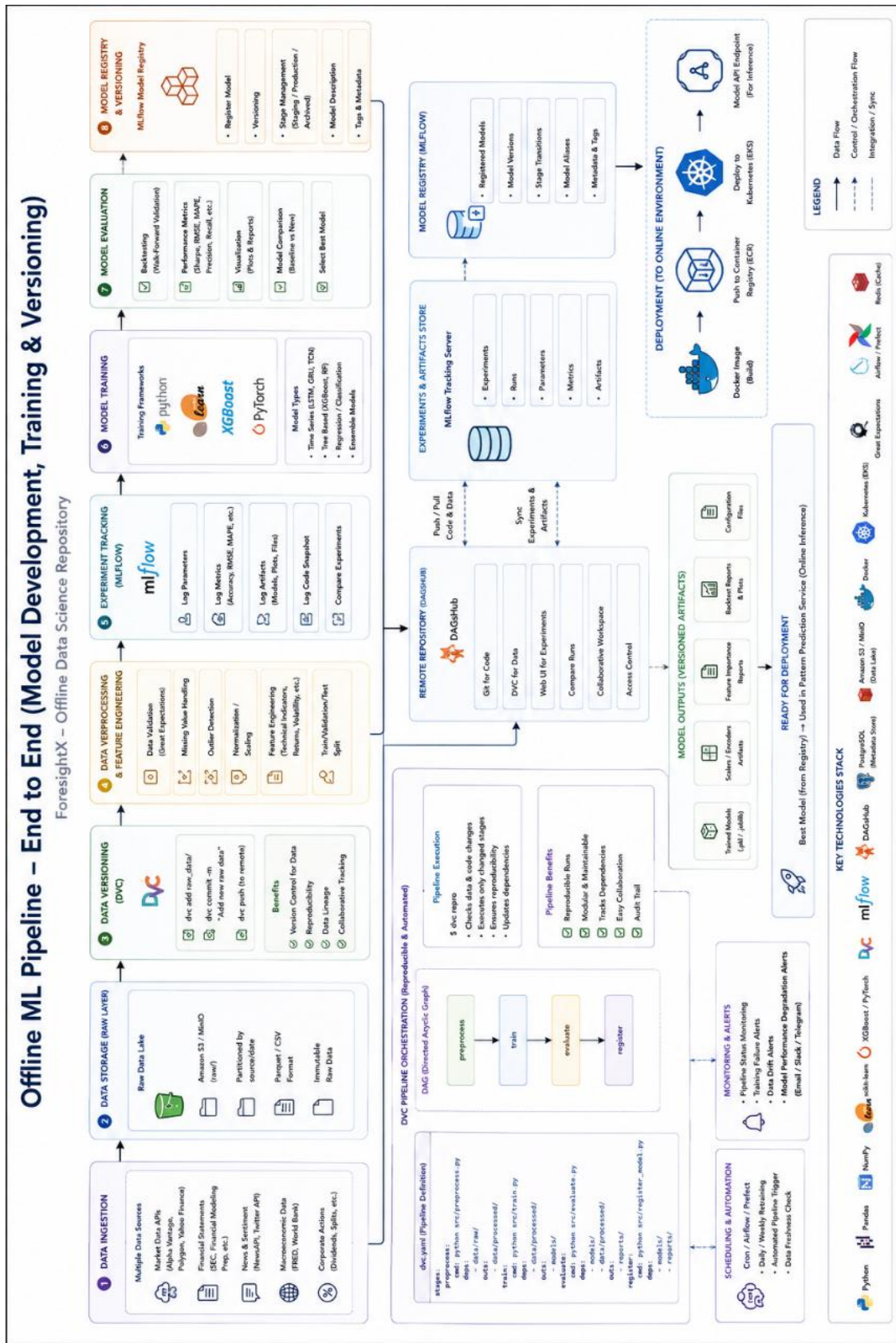
The second layer of the prediction service exposes inference endpoints that allow the orchestration service to request live predictions. The model returns multiple outputs instead of a single value. These outputs include next-hour prediction, next-day prediction, confidence score, and trend direction.

- Next-hour stock prediction
- Next-day stock prediction
- Day-after-next-day prediction
- Trend classification
- Confidence score generation

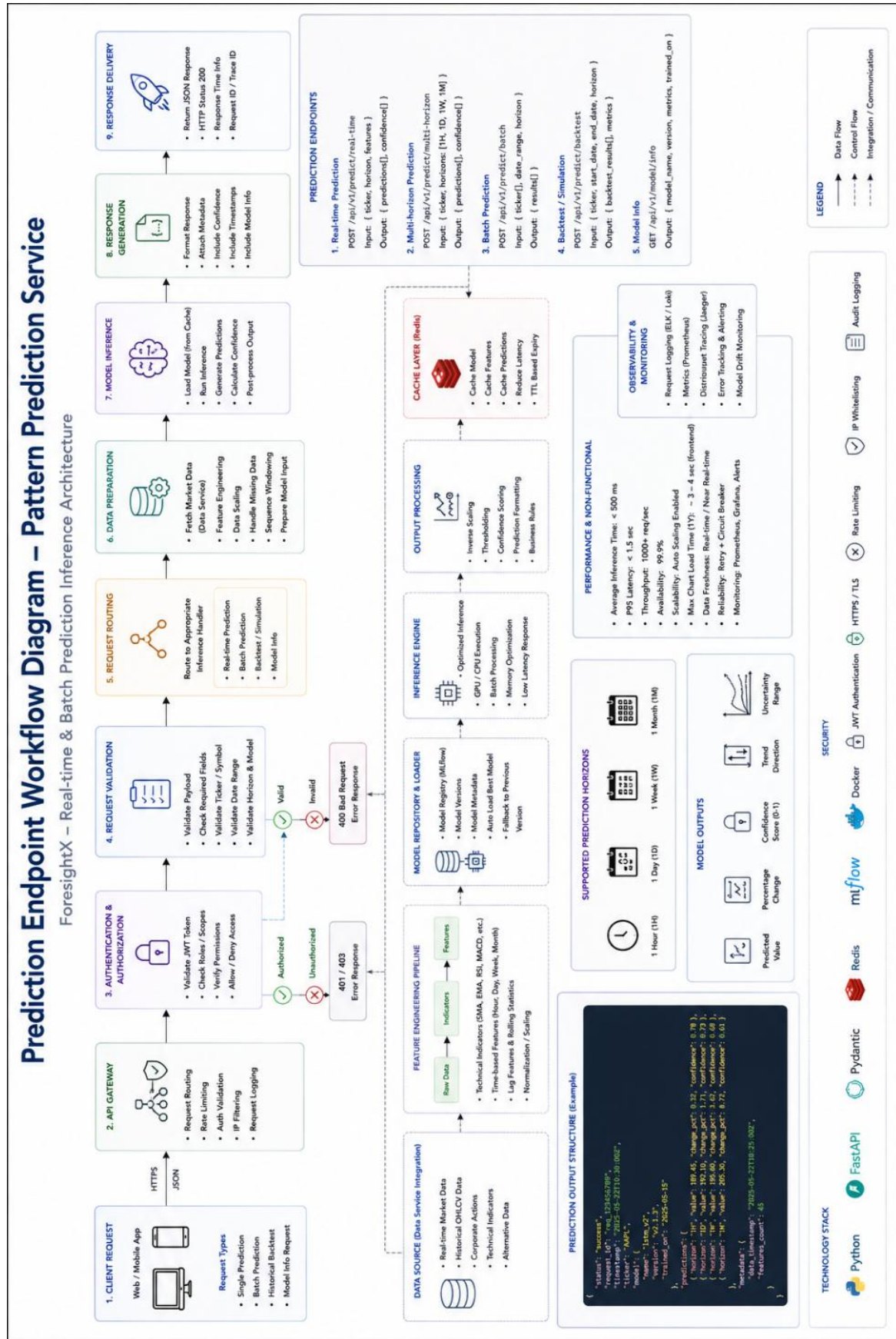
Authentication Service Workflow Diagram



Offline ML Pipeline Diagram



Prediction Endpoint Workflow Diagram



8.6 Methodology of Data Service

The data service is responsible for market data retrieval, historical data access, technical indicators, instrument search, and websocket streaming. The service acts as the primary market information provider for the entire platform.

A significant feature of the data service is caching. Since market data retrieval from external APIs can introduce latency, the service caches frequently accessed data to reduce repetitive requests and improve response times. This is one of the reasons the platform achieves acceptable chart-loading performance despite processing large historical datasets.

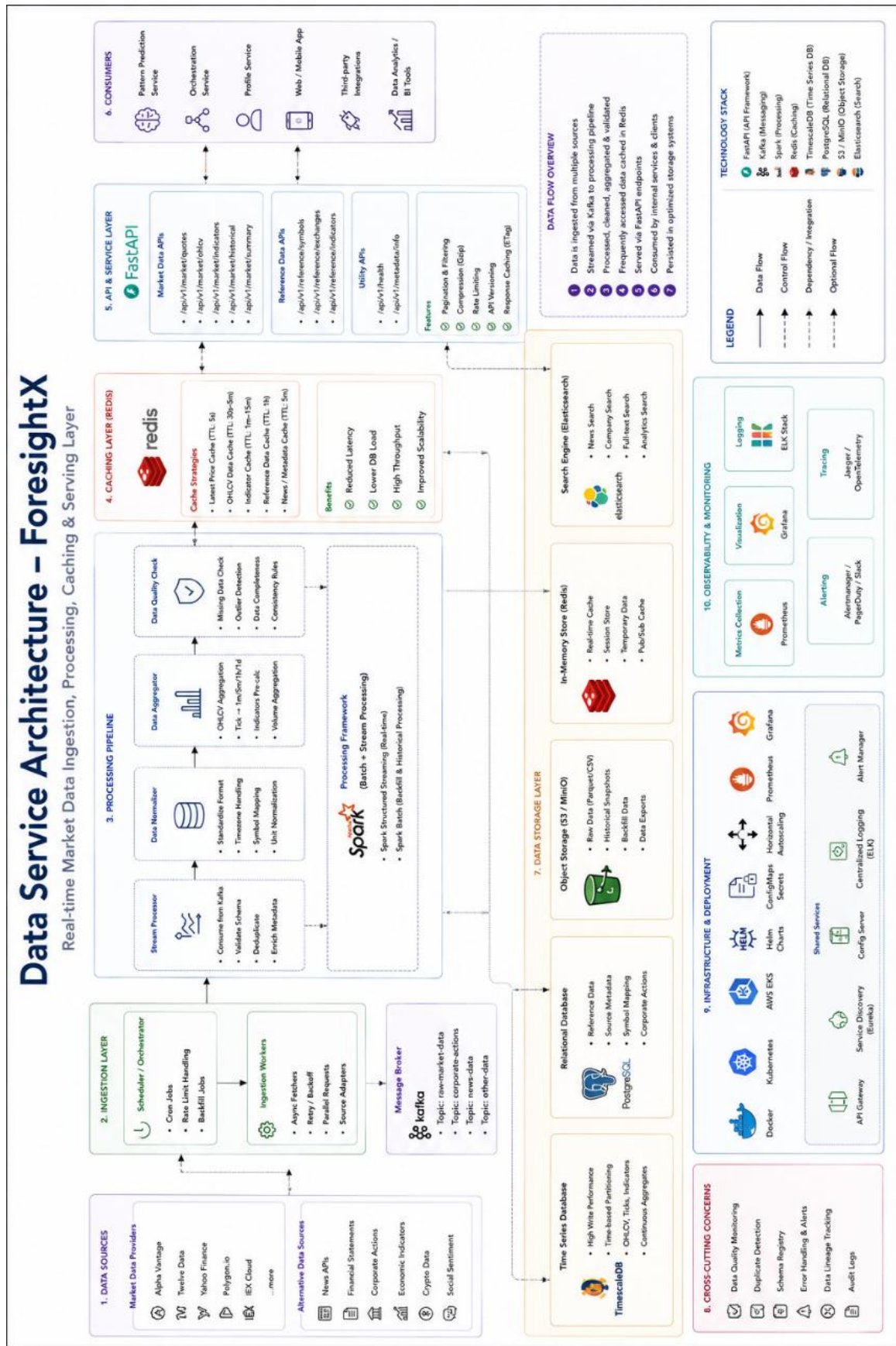
- Live stock price retrieval
- Historical data fetching
- Technical indicator computation
- Instrument search support
- Websocket streaming
- Caching layer for latency reduction

8.7 Methodology of Profile Service

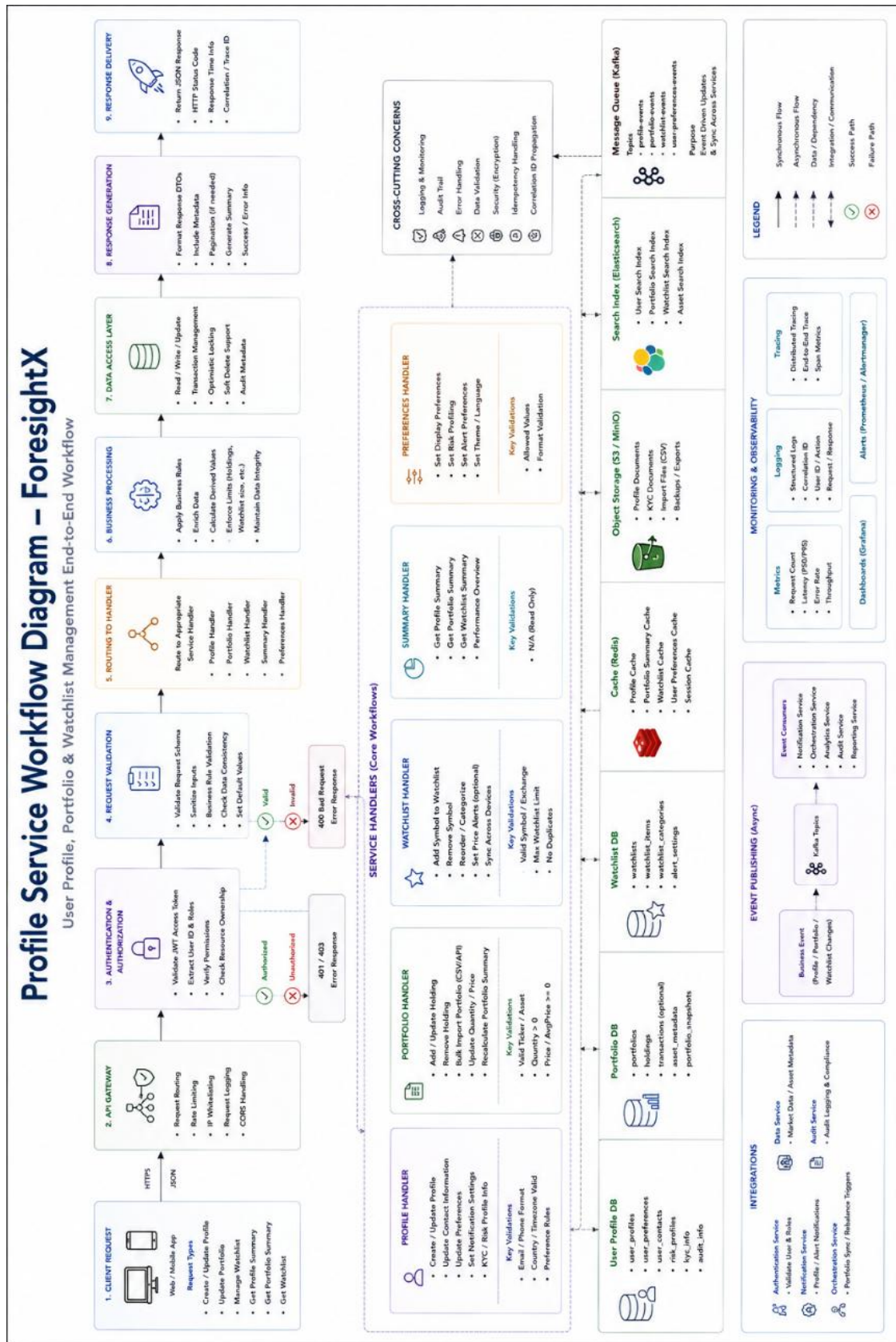
The profile service manages user-specific information such as portfolio holdings, profile details, and risk analysis. Unlike a simple profile database, the service attempts to provide contextual portfolio awareness.

The portfolio analysis mechanism evaluates the distribution of holdings and generates a simple risk-oriented interpretation. This makes the profile service useful not only for storage but also for recommendation personalization.

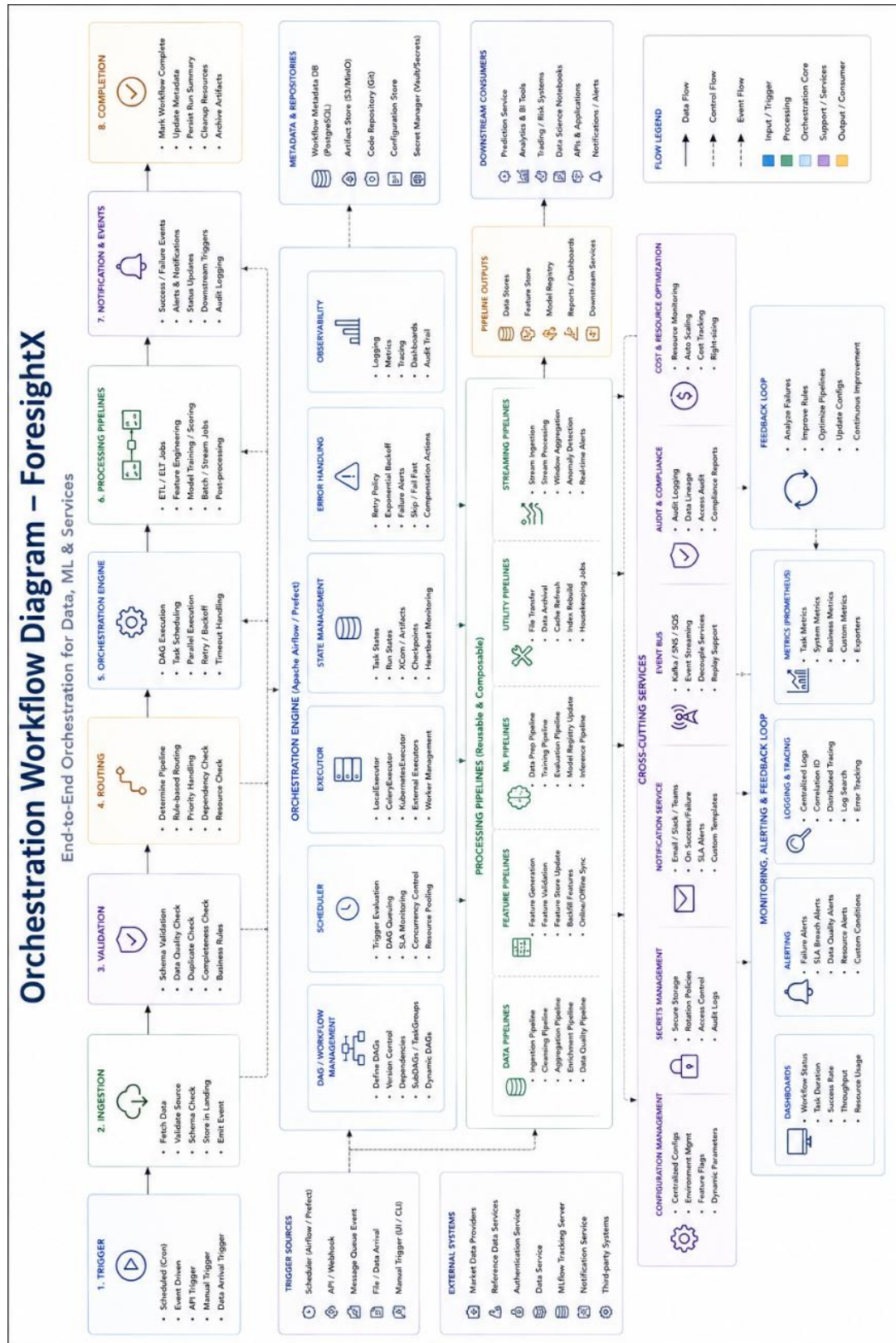
Data Service Architecture Diagram



Profile Service Workflow Diagram



Orchestration Workflow Diagram



8.8 Methodology of Orchestration Service

The orchestration service acts as the coordination layer of the complete platform. Instead of directly exposing raw outputs from every service, the orchestration layer combines data retrieval, prediction results, sentiment interpretation, and contextual logic into a unified response.

The orchestration workflow integrates LangChain, LangGraph, Gemini integration, and MCP-based dynamic interaction flow. The orchestration engine dynamically decides how much weight should be assigned to prediction outputs, sentiment analysis, and contextual indicators before generating the final recommendation.

This architecture is important because recommendation generation is not treated as a single-rule classification problem. Instead, it behaves as a multi-stage reasoning workflow where several independent outputs are combined before the final response is returned.

8.9 Recommendation Generation Algorithm

The recommendation generation workflow uses orchestration logic combined with Gemini-powered reasoning. The orchestration layer collects prediction results, technical indicators, and contextual signals before dynamically combining them into a recommendation.

- Collect prediction outputs
- Fetch technical indicators
- Analyze news sentiment
- Evaluate confidence score
- Apply weighted orchestration logic
- Generate AI-assisted recommendation summary

8.10 Deployment Workflow

The deployment workflow currently uses Docker-based containerization where every service executes in an isolated environment. The future infrastructure plan includes Kubernetes orchestration using EKS. Under this architecture, each microservice will execute inside separate Kubernetes pods.

Future CI/CD integration will automate testing, building, and deployment workflows. This approach improves scalability, consistency, and deployment reliability.

- Docker containerization
- Independent service deployment
- Future Kubernetes pod orchestration
- Planned CI/CD pipeline integration
- Cloud-ready scalable deployment workflow

9. Testing

9.1 Unit Testing

Unit testing was performed on individual microservices to verify endpoint behavior, token validation, database interactions, prediction response structure, and API-level business logic. Each service was tested independently before integration to ensure stable modular behavior.

9.2 System Testing

System testing validated the complete end-to-end workflow of the platform. This included authentication, stock search, chart retrieval, prediction generation, orchestration analysis, and portfolio management.

9.3 API Testing

API testing verified request-response consistency, schema validation, authentication middleware, error handling, and latency behavior across all services.

9.4 Test Cases

TC ID	Module	Test Case	Expected Result	Status
TC-01	Auth	User Registration	Account created	Pass
TC-02	Auth	Login	JWT generated	Pass
TC-03	Auth	Invalid Login	Unauthorized rejected	Pass
TC-04	Auth	Refresh Token	New token issued	Pass
TC-05	Data	Fetch Live Price	Live price returned	Pass
TC-06	Data	Historical Data	Chart data returned	Pass
TC-07	Data	Indicators	Indicators generated	Pass

TC-08	Data	Invalid Ticker	Error response returned	Pass
TC-09	Pattern	Prediction	Prediction generated	Pass
TC-10	Pattern	Confidence Score	Confidence returned	Pass
TC-11	Pattern	Trend Detection	Trend generated	Pass
TC-12	Orchestration	Analysis Workflow	Recommendation returned	Pass
TC-13	Profile	Portfolio Update	Portfolio updated	Pass
TC-14	Profile	Risk Analysis	Risk summary generated	Pass
TC-15	Frontend	Dashboard Load	Dashboard rendered	Pass
TC-16	Frontend	Chart Rendering	Charts loaded	Pass
TC-17	System	End-to-End Workflow	Complete flow validated	Pass
TC-18	System	Websocket Streaming	Real-time updates received	Pass
TC-19	API	Unauthorized Access	Request blocked	Pass
TC-20	Deployment	Docker Startup	Containers initialized	Pass

9.5 Debugging and Validation

Debugging involved validation of API schemas, orchestration logic, inter-service communication, and JWT propagation workflows. Major debugging activities included payload synchronization, dependency troubleshooting, and orchestration flow verification.

9.6 Testing Reports Screenshots

Test Report — ForesightX-orchestration

Date: 2026-05-13

Result

PASS — 8 tests passed.

Commands

```
# From repo root
.venv/bin/pytest -q --junitxml=../ForesightX-Docs/docs/reports/junit-foresightx-orchestration.xml
```

Summary

- Test files: 2
- Tests: 8
- Duration: ~6s

Artifacts

- JUnit XML: `ForesightX-Docs/docs/reports/junit-foresightx-orchestration.xml`

Notes

- Added `tests/conftest.py` to ensure `app` imports resolve when running `pytest` from the service root.

Test Report — ForesightX-Pattern

Date: 2026-05-13

Result

PASS — 5 tests passed.

Commands

```
# From repo root
../.venv/bin/python -m pytest -q --junitxml=../ForesightX-Docs/docs/reports/junit-foresightx-pattern.xml

# Reduced runtime file-set smoke test
FORESIGHTX_ARTIFACTS_DIR=/tmp/.../app/artifacts/model python -c "from foresightx_pattern.app.main import create_app; create_app()"
```

Summary

- Test files: 5
- Tests: 5
- Duration: ~12s
- Reduced runtime import smoke test: PASS

Artifacts

- JUnit XML: `ForesightX-Docs/docs/reports/junit-foresightx-pattern.xml`

10. Conclusion

ForesightX successfully demonstrates the implementation of a microservice-based intelligent stock analytics and prediction platform integrating distributed systems, machine learning workflows, orchestration logic, and modern frontend technologies.

The architecture separates authentication, prediction, orchestration, profile handling, and market data retrieval into independent services. This improves scalability, maintainability, debugging capability, and deployment flexibility.

The project also integrates MLOps-oriented practices through DVC, MLflow, and DAGsHub integration. Instead of treating the ML model as an isolated experiment, the project treats the prediction workflow as a reproducible engineering pipeline.

The orchestration layer significantly improves the usability of the system by combining prediction outputs, technical indicators, contextual analysis, and AI-assisted reasoning into structured recommendation summaries.

Future work includes Kubernetes orchestration, CI/CD automation, centralized monitoring, distributed tracing, and improved large-scale deployment support. Despite being developed as an academic project, the system demonstrates several production-oriented software engineering concepts. Educational institutions continue calling this a 'minor project' while students quietly build distributed cloud systems with ML pipelines attached to them like emotionally unstable satellites.

11. SWOT Analysis

11.1 Strengths

- The project uses a modular microservice architecture that improves maintainability and scalability.
- The orchestration workflow combines prediction outputs, contextual indicators, and AI-assisted reasoning.
- The use of JWT authentication improves distributed security handling.
- The integration of DVC, MLflow, and DAGsHub improves reproducibility and experiment tracking.
- The architecture reflects modern distributed software engineering practices.

11.2 Weaknesses

- The prediction model currently focuses on short-term forecasting only.
- The current deployment workflow is primarily Docker-based and Kubernetes support remains part of future development.
- The system depends on external market data APIs and third-party services.
- Microservice orchestration increases debugging complexity and integration overhead.

11.3 Opportunities

- The system can be extended toward Kubernetes-based cloud-native deployment.
- Advanced transformer-based financial prediction models can be integrated in future versions.
- The platform can evolve toward mobile applications, real-time alert systems, and advanced dashboards.
- Centralized monitoring and distributed tracing can significantly improve observability.

11.4 Threats

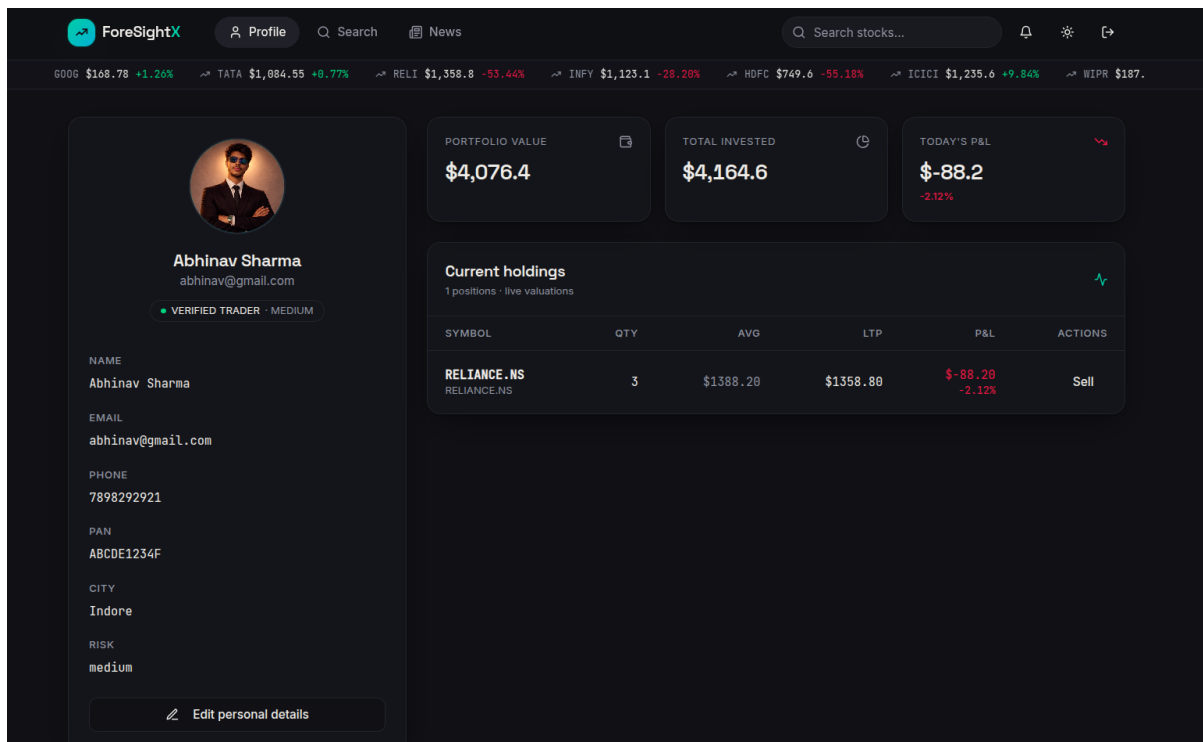
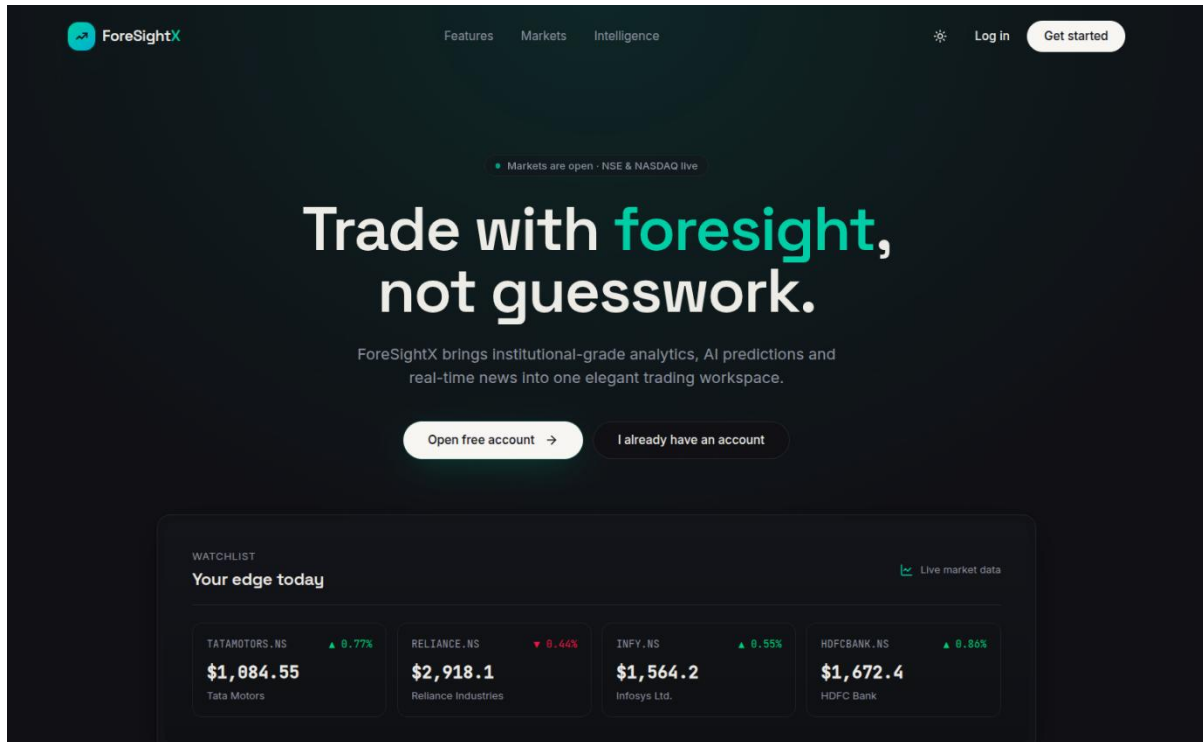
- Financial market volatility can reduce prediction reliability.
- Third-party API limitations or pricing changes may affect system behavior.
- Security vulnerabilities in distributed systems can become operational risks.
- Large-scale enterprise financial platforms provide strong commercial competition.

12. References

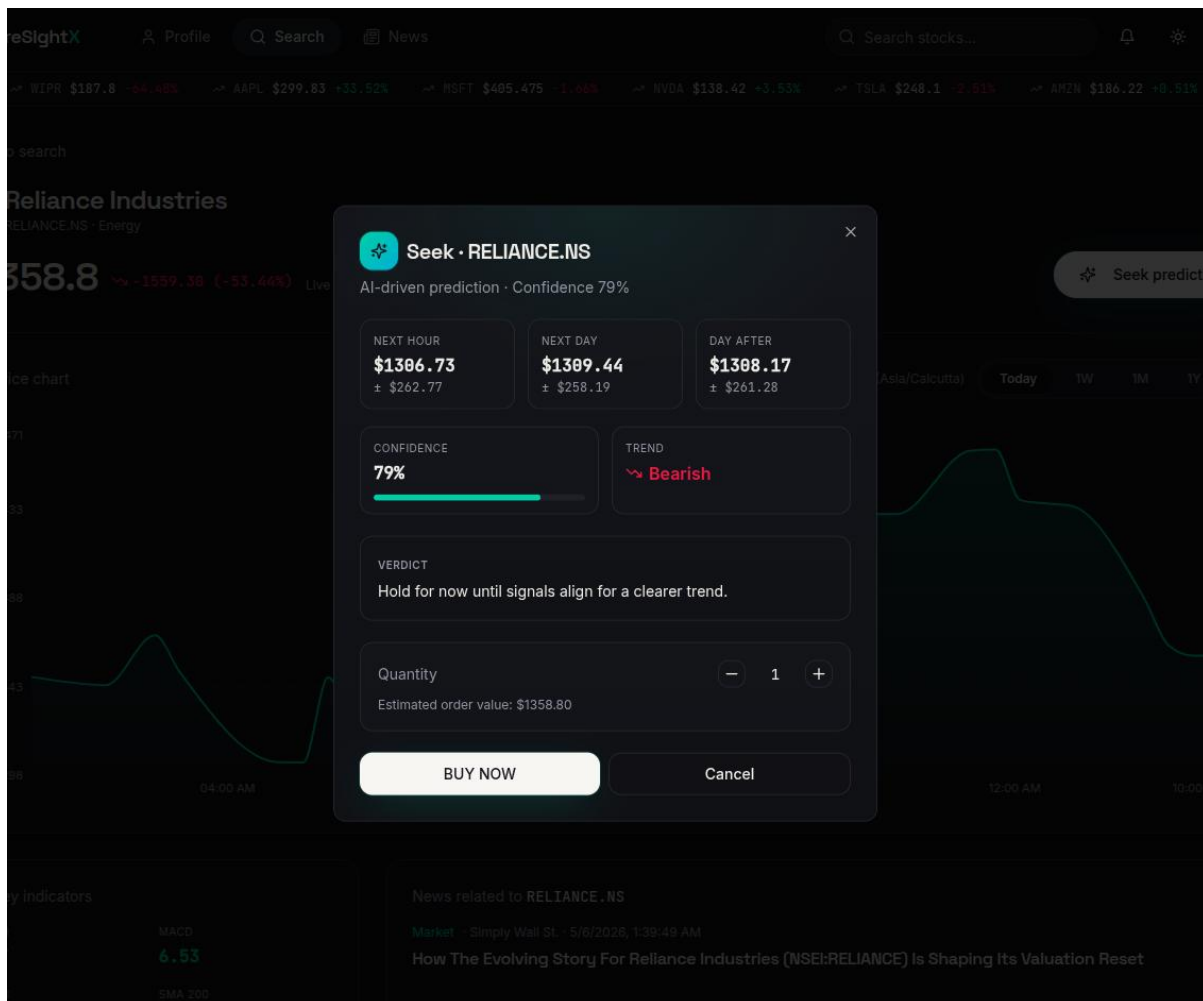
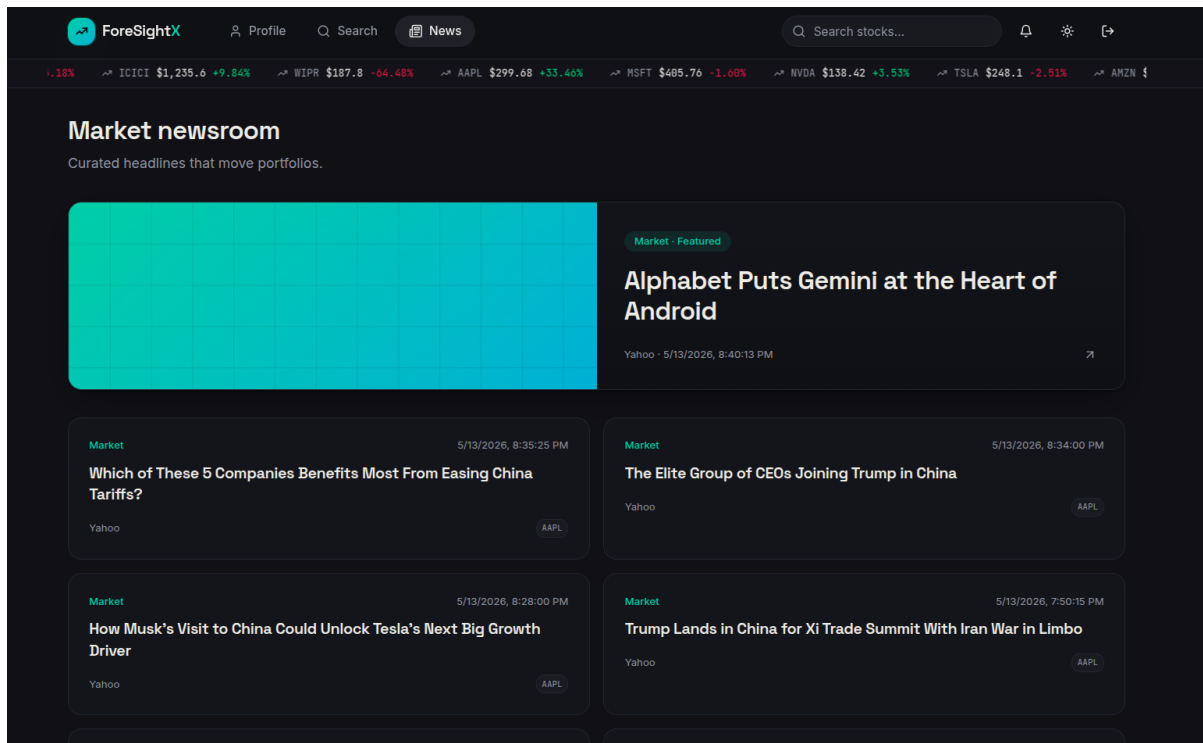
- Y. LeCun, Y. Bengio, and G. Hinton, “Deep Learning,” Nature, 2015.
- Goodfellow, Y. Bengio, and A. Courville, Deep Learning, MIT Press, 2016.
- FastAPI Documentation – <https://fastapi.tiangolo.com/>
- PyTorch Documentation – <https://pytorch.org/docs/>
- Scikit-learn Documentation – <https://scikit-learn.org/>
- Docker Documentation – <https://docs.docker.com/>
- MLflow Documentation – <https://mlflow.org/docs/latest/index.html>
- DVC Documentation – <https://dvc.org/doc>
- React Documentation – <https://react.dev/>
- PostgreSQL Documentation – <https://www.postgresql.org/docs/>
- Kubernetes Documentation – <https://kubernetes.io/docs/>
- AWS EC2 Documentation – <https://docs.aws.amazon.com/ec2/>
- JWT RFC 7519 Standard Documentation

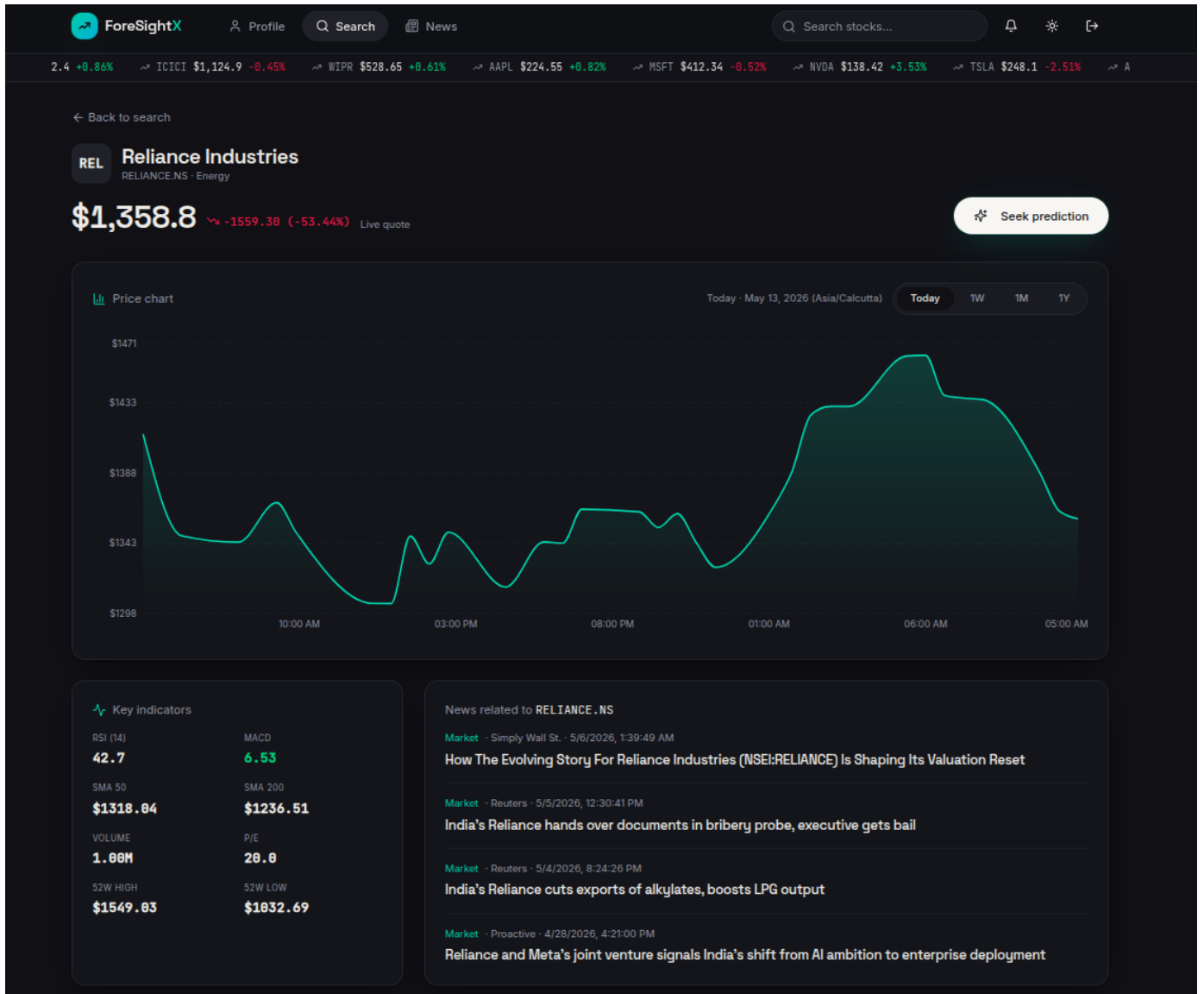
Appendix

A. Screenshots of Application



ForesightX - Minor Project Report





ForesightX - Minor Project Report

TheAditya-10 / ForesightX

Connected to - 15 hours ago

Unwatch 1 Star 1 Fork

Files Datasets Experiments 19 Models Annotations Collaboration Settings

Experiments < pine elephant

Delete

- Details
- Parameters
- Metrics
- Models
- Charts
- Artifacts

Details

Status: Finished

Commit ID: 67ba78883f

Source: <>

Data:

Date: a few seconds ago

Labels: +

Parameters	
Name	Value
dagshub_storage > enabled	true
dagshub_storage > paths > features	data/features/
dagshub_storage > paths > models	models/
dagshub_storage > repo	ForesightX
dagshub_storage > save_to_dagshub	true
dagshub_storage > username	TheAditya-10
dvc > cache_dir	.dvc/cache

Metrics	
Name	Value
No Rows To Show	

TheAditya-10 / ForesightX

Connected to - 15 hours ago

Unwatch 1 Star 1 Fork

Files Datasets Experiments 19 Models Annotations Collaboration Settings

Compare Reset filters Delete Archive Labels Columns Go to MLflow UI Log Experiment

	Code	Data	Name	Commit	Created	Labels	models > m...	models > tr...	logging > lo...	iterations	training_time	val_mae
☐	<>	@	pine elephant	67ba	2 months ago	+	200	0.0001	logs			
☐	<>	@	Register_MLP...	a912	5 months ago	+				200	17.967415	0.1516059785...
☐	<>	@	MLP_Eval_N...	a912	5 months ago	+						0.1516059785...
☐	<>	@	shadow turtle	a912	5 months ago	+	200		logs			
☐	<>	@	lake sheep	4c4e	5 months ago	+	200	0.0001				
☐	<>	@	Register_MLP...	7aae	5 months ago	+				200	38.617621	0.1683177872...
☐	<>	@	MLP_Eval_N...	7aae	5 months ago	+						0.1683177872...
☐	<>	@	violet puma	0075	5 months ago	+		0.0001				
☐	<>	@	Register_MLP...	7e7c	5 months ago	+				200	9.548441	0.0768431499...
☐	<>	@	shadow ox	7e7c	5 months ago	+		0.0001				
☐	<>	@	voice tapir	d1b8	5 months ago	+	200	0.0001	logs			
☐	<>	@	Register_MLP...	0e15	5 months ago	+				200	9.548441	0.0768431499...
☐	<>	@	MLP_Eval_TA...	0e15	5 months ago	+						0.0768431499...
☐	<>	@	Register_MLP...	0e15	5 months ago	+				176	9.390529	0.1189830955...
☐	<>	@	MLP_Eval_A...	0e15	5 months ago	+						0.1189830955...

B. API Endpoints

Authentication Service

- POST /auth/sign-up
- POST /auth/sign-in
- POST /auth/token/refresh
- POST /auth/sign-out
- GET /auth/me
- GET /oauth/google/authorize
- GET /oauth/google/callback
- GET /health

Data Service

- GET /price/{ticker}
- GET /history/{ticker}?limit=30
- GET /bars/{ticker}?interval=1h&limit=240
- GET /indicators/{ticker}
- GET /news/{ticker}
- GET /instruments/search?q=...&limit=15
- POST /stream/ingest
- WS /stream/{ticker}
- GET /health

Pattern Prediction Service

- POST /predict
- GET /health

Profile Service

- GET /portfolio/{user_id}
- POST /portfolio/update
- GET /portfolio/{user_id}/history
- GET /profiles/{user_id}
- PATCH /profiles/{user_id}

- POST /profiles/{user_id}/photo
- POST /profiles
- GET /risk/{user_id}
- GET /health

Orchestration Service

- POST /analyze
- GET /analysis/jobs
- GET /analysis/jobs/{job_id}
- GET /instruments/search?q=...&limit=15
- GET /health